

OPERATOR LEARNING: ALGORITHMS AND ANALYSIS

NIKOLA B. KOVACHKI, SAMUEL LANTHALER, AND ANDREW M. STUART

ABSTRACT. Operator learning refers to the application of ideas from machine learning to approximate (typically nonlinear) operators mapping between Banach spaces of functions. Such operators often arise from physical models expressed in terms of partial differential equations (PDEs). In this context, such approximate operators hold great potential as efficient surrogate models to complement traditional numerical methods in many-query tasks. Being data-driven, they also enable model discovery when a mathematical description in terms of a PDE is not available. This review focuses primarily on neural operators, built on the success of deep neural networks in the approximation of functions defined on finite dimensional Euclidean spaces. Empirically, neural operators have shown success in a variety of applications, but our theoretical understanding remains incomplete. This review article summarizes recent progress and the current state of our theoretical understanding of neural operators, focusing on an approximation theoretic point of view.

1. INTRODUCTION

This paper overviews algorithms and analysis related to the subject of operator learning: finding approximations of maps between Banach spaces, from data. Our focus is primarily on neural operators, which leverage the success of neural networks in finite dimensions; but we also cover related literature in the work specific to learning linear operators, and the use of Gaussian processes and random features. In subsection 1.1 we discuss the motivation for our specific perspective on operator learning. Subsection 1.2 contains a literature review. Subsection 1.3 overviews the remainder of the paper.

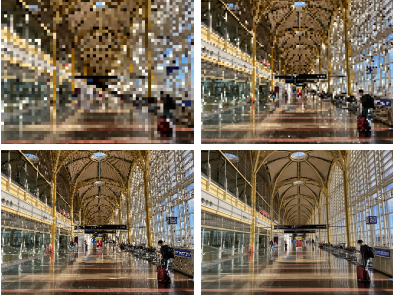
1.1. High Dimensional Vectors Versus Functions. Many tasks in machine learning require operations on high dimensional tensors¹ arising, for example, from pixellation of images or from discretization of a real-valued mapping defined over a subset of $\mathbb{R}^d$. The main idea underlying the work that we overview in this paper is that it can be beneficial, when designing and analyzing algorithms in this context, to view these high dimensional vectors as functions $u : \mathfrak{D} \rightarrow \mathbb{R}^c$ defined on a domain $\mathfrak{D} \subset \mathbb{R}^d$. For example $(c, d) = (3, 2)$ for RGB images and $(c, d) = (1, 3)$ for a scalar field such as temperature in a room. Pixellation, or discretization, of $\mathfrak{D}$ will lead to a tensor with size scaling like N , the number of pixels or discretization points; N will be large and hence the tensor will be of high dimension. Working with data-driven algorithms designed to act on function u , rather than the high dimensional tensor, captures intrinsic properties of the problem, and not details related to specific pixellation or discretization; as a consequence models learned from data can be transferred from one pixellation or discretization level to another.

Consider the image shown in Figure 1a, at four levels of resolution. As an RGB image it may be viewed as a vector in $\mathbb{R}^{3N}$ where N is the number of pixels. However by the time we reach the highest resolution (bottom right) it is more instructive to view it as a function mapping $\mathfrak{D} := [0, 1]^2 \subset \mathbb{R}^2$ into $\mathbb{R}^3$. This idea is summarized

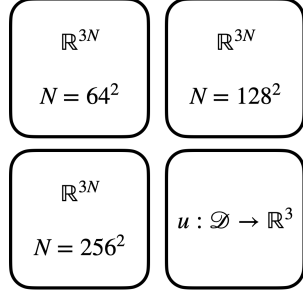
Date: February 27, 2024.

¹“Tensor” here may be a vector, matrix or object with more than two indices.

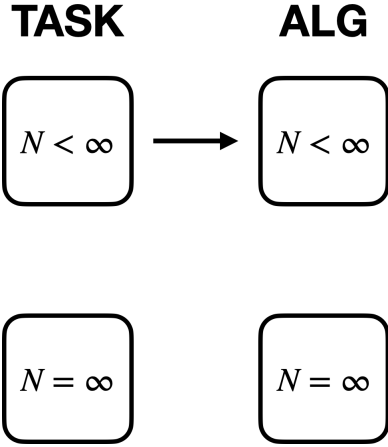
in Figure 1b. Even if the original machine learning task presents as acting on high dimensional tensor of dimension proportional to N , it is worth considering whether it may be formulated in the continuum limit $N \rightarrow \infty$, conceiving of algorithms in this setting, and only then approximating to finite dimension again to obtain practical algorithms. These ideas are illustrated in Figures 2a and 2b.



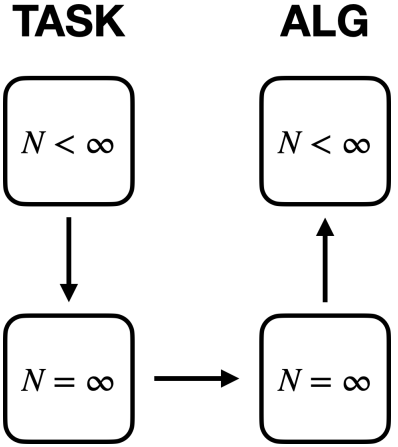
(A) Same images at different resolutions



(B) Different resolutions as vectors and (bottom right) as a function



(A) Directly design algorithm at fixed resolution N



(B) Design algorithm at limit of infinite resolution

1.2. Literature Review. We give a brief overview of the literature in this field; greater depth, and more citations, are given in subsequent sections.

Algorithms on Function Space. The idea of conceiving algorithms in the continuum, and only then discretizing, is prevalent in numerous areas of computational science and engineering. For example in the field of PDE constrained optimization the relative merits of optimize-then-discretize, in comparison with discretize-then-optimize, are frequently highlighted [54]. In the field of Bayesian inverse problems [61] formulation on Banach space [133] leads to new perspectives on algorithms, for example in MAP estimation [63]. And sampling probability measures via MCMC can be formulated on Banach spaces [32], leading to provably dimension independent convergence rates [51].

Supervised Learning on Function Space. Supervised learning [49] rose to prominence in the context of the use of deep neural network (DNN) methods for classifying digits and then images. In such contexts the task is formulated as learning a mapping from a Euclidean space (pixellated image) to a set of finite cardinality. Such methods are readily generalized to regression in which the output space is also a Euclidean space. However, applications in science and engineering, such as surrogate modeling [121] and scientific discovery [113], often suggest supervised learning tasks in which input and/or output spaces are infinite dimensional; in particular they comprise spaces of functions defined over subsets of Euclidean space. We refer to the resulting methods, conceived to solve supervised learning tasks where the inputs and outputs are functions, as neural operators. Whilst there is earlier work on regression in function space [116], perhaps the earliest paper to conceive of neural network-based supervised learning between spaces of functions is [24]. This work was generalized in the seminal DeepONet paper [92]. Concurrently with the development of DeepONet other methods were being developed, including methods based on model reduction [14] (PCA-Net) and on random features [101]. The random feature approach in [101] included the use of manipulations in the Fourier domain, to learn the solution operator for viscous Burgers’ equation, whose properties are well-understood in Fourier space. We also mention a related Fourier-based approach in [106,107]. The idea of using the Fourier transform was exploited more systematically through development of the Fourier Neural Operator (FNO) [83]. The framework introduced in this paper has subsequently been generalized to work with sets of functions other than Fourier, such as wavelets [135], spherical harmonics [16] and more general sets of functions [13,73]. The FNO architecture is related to convolutional neural networks, which have also been explored for operator learning, see e.g. [45,88,117,118], and [50,115] for relevant work. We mention also similar developments in computer graphics where, in [104], a method is proposed based on projections onto the eigenfunctions of the Laplace-Beltrami operator and is subsequently extended in [89,127,146]. At a more foundational level, recent work [11] develops a theoretical framework to study neural operators, aiming to pinpoint theoretical distinctions between these infinite-dimensional architectures from conventional finite-dimensional approaches, based on a frame-theoretic notion of representation equivalence.

Approximation Theory. The starting point for approximation theory is universal approximation. Such theory is overviewed in the finite dimensional setting in [109]. It is developed systematically for neural operators in [66], work that also appeared in [68]. However, the first paper to study universal approximation, in the context of mappings between spaces of scalar-valued functions, is Chen and Chen [24]. This was followed by work extending their analysis to DeepONet [74], analysis for the FNO [67] and analysis for PCA-Net in [14], and a number of more recent contributions, e.g. [21,22,56,57,60,149].

The paper [92] first introduced DeepONets and studied their practical application on a number of prototypical problems involving differential equations. The empirical paper [35] studies various neural operators from the perspective of the cost-accuracy trade-off, studying how many parameters, or how much data, is needed to achieve a given error. Such complexity issues are studied theoretically for DeepONet, in the context of learning the solution operator for the incompressible Navier-Stokes equation and several other PDEs, in [74], with analogous analysis for PCA-Net in [72]. In [94] the coefficient to solution map is studied for divergence form elliptic PDEs, and analyticity of the coefficient and the solution is exploited to study complexity of the resulting neural operators. The paper [52] studies complexity for the same problem, but exploits operator holomorphy. In [78] complexity

is studied for Hamilton-Jacobi equations, using approximation of the underlying characteristic flow. The work [46] discusses conditions under which neural operator layers are injective and surjective. The sample complexity of operator learning with DeepONet and related architectures is discussed in [90]. Out-of-distribution bounds are discussed in [13].

The paper [36] studies the learning of linear operators from data. This subject is developed for elliptic and parabolic equations, and in particular for the learning of Greens functions, in [17–19, 48, 132, 137] and, for spectral properties of the Koopman operator, the solution operator for advection equations, in [31, 65].

1.3. Overview of Paper. In section 2 we introduce operator learning as a supervised learning problem on Banach space; we formulate testing and training in this context, and provide an example from porous medium flow. Section 3 is devoted to definitions of the supervised learning architectures that we focus on in this paper: PCA-Net, DeepONet, the Fourier Neural Operator (FNO) and random features methods. Section 4 describes various aspects of universal approximation theories in the context of operator learning. In section 5 we study complexity of these approximation, including discussion of linear operator learning; specifically we study questions such as how many parameters, or how much data, is required to achieve an operator approximation with a specified level of accuracy; and what properties of the operator can be exploited to reduce complexity? We summarize and conclude in section 6.

2. OPERATOR LEARNING

In subsection 2.1 we define supervised learning, followed in subsection 2.2 by discussion of the topic in the specific case of operator learning. Subsection 2.3 is devoted to explaining how the approximate operator is found from data (training) and how it is evaluated (testing). Subsection 2.4 describes how latent structure can be built into operator approximation, and learned from data. Subsection 2.5 contains an example from parametric partial differential equations (PDEs) describing flow in a porous medium.

2.1. Supervised Learning. The objective of supervised learning is to determine an underlying mapping $\Psi^\dagger : \mathcal{U} \rightarrow \mathcal{V}$ from samples²

$$(2.1) \quad \{u_n, \Psi^\dagger(u_n)\}_{n=1}^N, \quad u_n \sim \mu.$$

Here the probability measure μ is supported on $\mathcal{U}$. Often supervised learning is formulated by use of the data model

$$(2.2) \quad \{u_n, v_n\}_{n=1}^N, \quad (u_n, v_n) \sim \pi,$$

where the probability measure π is supported on $\mathcal{U} \times \mathcal{V}$. The data model (2.1) is a special case which is sufficient for the exposition in this article.

In the original applications of supervised learning $\mathcal{U} = \mathbb{R}^{d_x}$ and $\mathcal{V} = \mathbb{R}^{d_y}$ (regression) or $\mathcal{V} = \{1, \dots, K\}$ (classification). We now go beyond this setting.

2.2. Supervised Learning of Operators. In many applications arising in science and engineering it is desirable to consider a generalization of the finite-dimensional setting to separable Banach spaces $\mathcal{U}, \mathcal{V}$ of vector-valued functions:

$$\begin{aligned} \mathcal{U} &= \{u : \mathfrak{D}_x \rightarrow \mathbb{R}^{d_i}\}, \quad \mathfrak{D}_x \subseteq \mathbb{R}^{d_x} \\ \mathcal{V} &= \{v : \mathfrak{D}_y \rightarrow \mathbb{R}^{d_o}\}, \quad \mathfrak{D}_y \subseteq \mathbb{R}^{d_y}. \end{aligned}$$

²Note that from now on N denotes the data volume (and not the size of a finite dimensional problem as in subsection 1.1).

Given data (2.1) we seek to determine an approximation to $\Psi^\dagger : \mathcal{U} \rightarrow \mathcal{V}$ from within a family of parameterized functions

$$\Psi : \mathcal{U} \times \Theta \mapsto \mathcal{V}.$$

Here $\Theta \subseteq \mathbb{R}^p$ denotes the parameter space from which we seek the optimal choice of parameter, denoted θ^* . Parameter θ^* may be chosen in a data-driven fashion to optimize the approximation of $\Psi^\dagger$ by $\Psi(\cdot; \theta^*)$; see the next subsection. In section 4 we will discuss the choice of θ^* from the perspective of approximation theory.

2.3. Training and Testing. The data (2.1) is used to train the model Ψ ; that is, to determine a choice of θ . To this end we introduce an error, or relative error, measure $r : \mathcal{V}' \times \mathcal{V}' \rightarrow \mathbb{R}^+$. Here $\mathcal{V}'$ is another Banach space containing the range of $\Psi^\dagger$ and $\Psi(\cdot; \theta)$. Typical choices for r include the error

$$(2.3) \quad r(v_1, v_2) = \|v_1 - v_2\|_{\mathcal{V}'}$$

and, for $\varepsilon \in (0, \infty)$, one of the relative errors

$$(2.4) \quad r(v_1, v_2) = \frac{\|v_1 - v_2\|_{\mathcal{V}'}}{\varepsilon + \|v_1\|_{\mathcal{V}'}} \quad \text{or} \quad r(v_1, v_2) = \frac{\|v_1 - v_2\|_{\mathcal{V}'}}{\max\{\varepsilon, \|v_1\|_{\mathcal{V}'}\}}.$$

Now let μ_N be the empirical measure

$$\mu_N = \frac{1}{N} \sum_{n=1}^N \delta_{u_n}.$$

Then the parameter θ^* is determined from

$$\theta^* = \operatorname{argmin}_{\theta} \mathcal{R}_N(\theta), \quad \mathcal{R}_N(\theta) := \mathbb{E}^{u \sim \mu_N} [r(\Psi^\dagger(u), \Psi(u; \theta))^q],$$

for some positive q , typically $q = 1$. Function $\mathcal{R}_N(\theta)$ is known as the empirical risk; also of interest is the expected (or population) risk

$$\mathcal{R}_\infty(\theta) := \mathbb{E}^{u \sim \mu} [r(\Psi^\dagger(u), \Psi(u; \theta))^q].$$

Note that $\mathcal{R}_\infty(\theta)$ requires knowledge of data in the form of the entire probability measure μ .

Once trained, models are typically tested by evaluating the error

$$\text{error} := \mathbb{E}^{u \sim \mu'} [r(\Psi^\dagger(u), \Psi(u; \theta))^q].$$

Here μ' is defined on the support of μ . For computational purposes the measure μ' is often chosen as another empirical approximation of μ , independently of μ_N ; other empirical measures may also be used. For theoretical analyses μ' may be chosen equal to μ , but other choices may also be of interest in determining the robustness of the learned model; see, for example, [13].

2.4. Finding Latent Structure. Behind many neural operators is the extraction of latent finite dimensional structure, as illustrated in Figure 3. Here we have two encoder/decoder pairs on $\mathcal{U}$ and $\mathcal{V}$, namely

$$G_{\mathcal{U}} \circ F_{\mathcal{U}} \approx I_{\mathcal{U}}, \quad G_{\mathcal{V}} \circ F_{\mathcal{V}} \approx I_{\mathcal{V}}$$

where $I_{\mathcal{U}}, I_{\mathcal{V}}$ are the identity maps on $\mathcal{U}$ and $\mathcal{V}$ respectively. Then φ is chosen so that

$$G_{\mathcal{V}} \circ \varphi \circ F_{\mathcal{U}} \approx \Psi^\dagger.$$

The map $F_{\mathcal{U}}$ extracts a finite dimensional latent space from the input Banach space while the map $G_{\mathcal{V}}$ returns from a second finite dimensional latent space to the output Banach space. These encoder-decoder pairs can be learned, reducing the operator approximation to a finite dimensional problem.

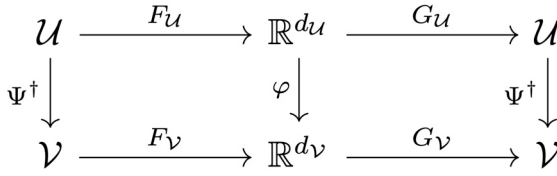
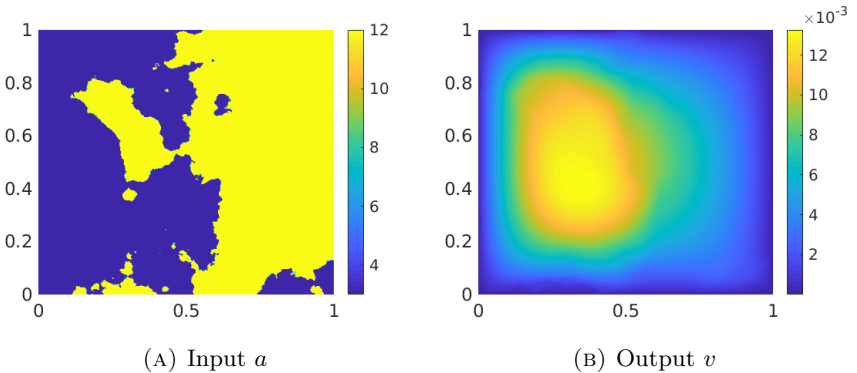


FIGURE 3. Latent Structure in Maps Between Banach Spaces $\mathcal{U}$ and $\mathcal{V}$

2.5. Example (Fluid Flow in a Porous Medium). We consider the problem of finding the piezometric head v from permeability a in a porous medium assumed to be governed by the Darcy Law in domain $\mathfrak{D} \subset \mathbb{R}^2$. This results in the need to solve the PDE

$$(2.5) \quad \begin{cases} -\nabla \cdot (a \nabla v) = f, & z \in \mathfrak{D} \\ v = 0, & z \in \partial \mathfrak{D}. \end{cases}$$

Here we consider $f \in H^{-1}(\mathfrak{D})$ to be given and fixed. The operator of interest $\Psi^\dagger : a \mapsto v$ then maps from a subset of the Banach space $L^\infty(\mathfrak{D})$ into $H_0^1(\mathfrak{D})$. An example of a typical input-output pair is shown in Figures 4a, 4b. Because the equation requires strictly positive a , in order to be well-defined mathematically and to be physically meaningful, the probability measure μ must be chosen carefully. Furthermore, from the point of view of approximation theory, it is desirable that the the space $\mathcal{U}$ is separable; for this reason it is often chosen to be $L^2(\mathfrak{D})$ and the measure supported on functions a satisfying a positive lower bound and a finite upper bound. Draws from such a measure are in $L^\infty(\mathfrak{D})$ and satisfy the necessary positivity and boundedness inequalities required for a solution to the Darcy problem to exist [44].



3. SPECIFIC SUPERVISED LEARNING ARCHITECTURES

Having reviewed the general philosophy behind operator learning, we next aim to illustrate how this methodology is practically implemented. To this end, we review several representative proposals for neural operator architectures, below.

³We use the notation a for input functions here, because it is a commonly adopted notation in applications to porous medium flow.

3.1. PCA-Net. The PCA-Net architecture was proposed as an operator learning framework in [14], anticipated in [53, 134]. In the setting of PCA-Net, $\Psi^\dagger : \mathcal{U} \rightarrow \mathcal{V}$ is an operator mapping between Hilbert spaces $\mathcal{U}$ and $\mathcal{V}$, and inputs are drawn from a probability measure μ on $\mathcal{U}$. Principal component analysis (PCA) is employed to obtain data-driven encoders and decoders, which are combined with a neural network mapping to give rise to the PCA-Net architecture.

The encoder is defined from PCA basis functions on the input space $\mathcal{U}$, computed from the covariance under μ : the encoder $F_{\mathcal{U}}$ is determined by projection onto the first $d_{\mathcal{U}}$ PCA basis functions $\{\phi_j\}_{j=1}^{d_{\mathcal{U}}}$. The encoding dimension $d_{\mathcal{U}}$ represents a hyperparameter of the architecture. The resulting encoder is given by a linear mapping to the PCA coefficients,

$$F_{\mathcal{U}} : \mathcal{U} \rightarrow \mathbb{R}^{d_{\mathcal{U}}}, \quad F_{\mathcal{U}}(u) = Lu := \{\langle \phi_j, u \rangle\}_{j=1}^{d_{\mathcal{U}}}.$$

The decoder $G_{\mathcal{V}}$ on $\mathcal{V}$ is similarly obtained from PCA under the the push-forward measure $\Psi_{\#}^\dagger \mu$. Denoting by $\{\psi_j\}_{j=1}^{d_{\mathcal{V}}}$ the first $d_{\mathcal{V}}$ basis functions under this push-forward, the PCA-Net decoder is defined by an expansion in this basis, i.e.

$$G_{\mathcal{U}} : \mathbb{R}^{d_{\mathcal{V}}} \rightarrow \mathcal{V}, \quad G_{\mathcal{U}}(\alpha) = \sum_{j=1}^{d_{\mathcal{V}}} \alpha_j \psi_j.$$

The PCA dimension $d_{\mathcal{V}}$ represents another hyperparameter of the PCA-Net architecture.

Finally, the PCA encoding and decoding on $\mathcal{U}$ and $\mathcal{V}$ are combined with a finite-dimensional neural network $\alpha : \mathbb{R}^{d_{\mathcal{U}}} \times \Theta \rightarrow \mathbb{R}^{d_{\mathcal{V}}}$, $w \mapsto \alpha(w; \theta)$ where

$$\alpha(w; \theta) := (\alpha_1(w; \theta), \dots, \alpha_{d_{\mathcal{V}}}(w; \theta)),$$

parametrized by $\theta \in \Theta$. This results in an operator $\Psi_{PCA} : \mathcal{U} \rightarrow \mathcal{V}$, of the form

$$\Psi_{PCA}(u; \theta)(y) = \sum_{j=1}^{d_{\mathcal{V}}} \alpha_j(Lu; \theta) \psi_j(y), \quad \forall u \in \mathcal{U}, \quad y \in \mathfrak{D}_y$$

which defines the PCA-Net architecture. Hyperparameters include the dimensions of PCA $d_{\mathcal{U}}$, $d_{\mathcal{V}}$, and additional hyperparameters determining the neural network architecture of α . In practice we are given samples of input-/output-function pairs with u_j sampled i.i.d. from μ :

$$\{(u_1, v_1), \dots, (u_N, v_N)\},$$

where $v_j := \Psi^\dagger(u_j)$. Then the PCA basis functions are determined from the covariance under an empirical approximation μ_N of μ , and its pushforward under $\Psi^\dagger$. The same data is then used to train neural network parameter $\theta \in \Theta$ defining $\alpha(w; \theta)$.

3.2. DeepONet. The DeepONet architecture was first proposed as a practical operator learning framework in [92], building on early work by Chen and Chen [24]. Similar to PCA-Net, the DeepONet architecture is also defined in terms of an encoder $F_{\mathcal{U}}$ on the input space, a finite-dimensional neural network α between the latent finite-dimensional spaces, and a decoder $G_{\mathcal{V}}$ on the output space. To simplify notation, we only summarize this architecture for real-valued input and output functions. Extension to operators mapping between vector-valued functions is straightforward.

The encoder in the DeepONet architecture is given by a general linear map L ,

$$F_{\mathcal{U}} : \mathcal{U} \rightarrow \mathbb{R}^{d_{\mathcal{U}}}, \quad F_{\mathcal{U}}(u) = Lu.$$

Popular choices for the encoding include a mapping to PCA coefficients, or could comprise pointwise observations $\{u(x_\ell)\}_{\ell=1}^{d_{\mathcal{U}}}$ at a pre-determined set of so-called

sensor points x_ℓ . Another alternative of note are active subspaces [147], which combine information about the input distribution and forward mapping through its gradient. This approach has been explored for function encoding in scientific ML in [93, 102].

The decoder in the DeepONet architecture is given by expansion with respect to a neural network basis. Given a neural network $\psi : \mathfrak{D}_y \times \Theta_\psi \rightarrow \mathbb{R}^{d_\psi}$, which defines a parametrized function from the domain $\mathfrak{D}_y$ of the output functions to $\mathbb{R}^{d_\psi}$, the DeepONet decoder is defined as

$$G_{\mathcal{V}} : \mathbb{R}^{d_\psi} \rightarrow \mathcal{V}, \quad G_{\mathcal{V}}(\alpha) = \sum_{j=1}^{d_\psi} \alpha_j \psi_j.$$

The above encoder and decoders on $\mathcal{U}$ and $\mathcal{V}$ are combined with a finite-dimensional neural network $\alpha : \mathbb{R}^{d_u} \times \Theta_\alpha \rightarrow \mathbb{R}^{d_\psi}$, to define the parametrized DeepONet,

$$\Psi_{DEEP}(u; \theta)(y) = \sum_{j=1}^{d_\psi} \alpha_j(Lu; \theta_\alpha) \psi_j(y; \theta_\psi), \quad \forall u \in \mathcal{U}, \quad y \in \mathfrak{D}_y.$$

This architecture is specified by choice of the linear encoding L , and choice of neural network architectures for α and ψ . Following [92] the network α is conventionally referred to as the “branch-net” (often denoted b or β), while ψ is referred to as the “trunk-net” (often denoted t or τ). The combined parameters $\theta = \{\theta_\alpha, \theta_\psi\}$ of these neural networks are learned from data of input- and output-functions.

3.3. FNO. The Fourier Neural Operator (FNO) architecture was introduced in [68, 83]. In contrast to the PCA-Net and DeepONet architectures above, FNO is not based on an approach that combines an encoding/decoding to a finite-dimensional latent space with a finite-dimensional neural network. Instead, neural operators such as the FNO generalize the structure of finite-dimensional neural networks to a function space setting, as summarized below. We will assume that the domain of the input and output functions $\mathfrak{D}_x = [0, 2\pi]^d$ can be identified with the d -dimensional periodic torus.

FNO is an operator architecture of the form,

$$\begin{aligned} \Psi_{FNO}(u; \theta) &= \mathcal{Q} \circ \mathcal{L}_L \circ \cdots \circ \mathcal{L}_2 \circ \mathcal{L}_1 \circ \mathcal{R}(u), \quad \forall u \in \mathcal{U}, \\ \mathcal{L}_\ell(v)(x; \theta) &= \sigma(W_\ell v(x) + b_\ell + \mathcal{K}(v)(x; \gamma_\ell)). \end{aligned}$$

It comprises of input and output layers $\mathcal{Q}, \mathcal{R}$, given by a pointwise composition with either a shallow neural network or a linear transformation, and several hidden layers $\mathcal{L}_\ell$.

Upon specification of a “channel width” d_c , the ℓ -the hidden layer takes as input a vector-valued function $v : x \mapsto v(x) \in \mathbb{R}^{d_c}$ and outputs another vector-valued function $\mathcal{L}_\ell(v) : x \mapsto \mathcal{L}_\ell(v)(x) \in \mathbb{R}^{d_c}$.⁴ Each hidden layer involves an affine transformation

$$v(x) \mapsto w(x) := W_\ell v(x) + b_\ell + \mathcal{K}(v)(x; \gamma_\ell),$$

and a pointwise composition with a standard activation function

$$w(x) \mapsto \sigma(w(x)),$$

where σ could e.g. be the rectified linear unit or a smooth variant thereof.

In the affine transformation, the matrix-vector pair (W_ℓ, b_ℓ) defines a pointwise affine transformation of the input $v(x)$, i.e. multiplication by matrix W_ℓ and adding

⁴Channel width can change from layer to layer; we simplify the exposition by fixing it.

a bias b_ℓ . $\mathcal{K}$ is a convolutional integral operator, parameterized by γ_ℓ ,

$$\mathcal{K}(v)(x; \gamma_\ell) = \int_{\mathfrak{D}_x} \kappa(x - y; \gamma_\ell) v(y) dy,$$

with $\kappa(\cdot; \gamma_\ell)$ a matrix-valued integral kernel. The convolutional operator can be conveniently evaluated via the Fourier transform $\mathcal{F}$, giving rise to a matrix-valued Fourier multiplier,

$$\mathcal{K}(v)(x; \gamma_\ell) = \mathcal{F}^{-1}(\mathcal{F}(\kappa(\cdot; \gamma_\ell))\mathcal{F}(v)),$$

where the Fourier transform is computed componentwise, and given by $\mathcal{F}(v)(k) = \int_{\mathfrak{D}_x} v(x) e^{-ik \cdot x} dx$. To be more specific, if we write $\kappa(x) = (\kappa_{ij}(x))_{i,j=1}^{d_c}$ in terms of its components, and if $\hat{\kappa}_{k,ij}$ denotes the k -th Fourier coefficient of $\kappa_{ij}(x)$, then the i -th component $\mathcal{K}(v)_i$ of the vector-valued output function $\mathcal{K}(v)$ is given by

$$[\mathcal{K}(v)_i](x; \gamma_\ell) = \frac{1}{(2\pi)^d} \sum_{k \in \mathbb{Z}^d} \left(\sum_{j=1}^{d_c} \hat{\kappa}_{k,ij} \mathcal{F}(v_j)(k) \right) e^{ik \cdot x}.$$

Here, the inner sum represents the action of $\hat{\kappa} = \mathcal{F}(\kappa)$ on $\mathcal{F}(v)$, and the outer sum is the inverse Fourier transform $\mathcal{F}^{-1}$. The Fourier coefficients $\hat{\kappa}_{k,ij}$ represent the tunable parameters of the convolutional operator. In a practical implementation, a Fourier cut-off $k_{\max}$ is introduced and the sum over k is restricted to Fourier wavenumbers $|k|_{\ell^\infty} \leq k_{\max}$, with $|\cdot|_{\ell^\infty}$ the ℓ^∞ -norm, resulting in a finite number of tunable parameters $\gamma_\ell = \{\hat{\kappa}_{k,ij} : |k|_{\ell^\infty} \leq k_{\max}, i, j = 1, \dots, d_c\}$.

To summarize, the FNO architecture is defined by

- (1) an input layer $\mathcal{R}$, given by pointwise composition of the input function with a shallow neural network or a linear transformation,
- (2) hidden layers $\mathcal{L}_1, \dots, \mathcal{L}_L$ involving, for each $\ell = 1, \dots, L$, matrix W_ℓ , bias b_ℓ and convolutional operator $\mathcal{K}(\cdot; \gamma_\ell)$ with parameters γ_ℓ identified with the corresponding Fourier multipliers $\hat{\kappa}_{k,ij}$,
- (3) an output layer $\mathcal{Q}$, given by pointwise composition with a shallow neural network or a linear transformation.

The composition of these layers defines a parametrized operator $u \mapsto \Psi_{\text{FNO}}(u; \theta)$, where θ collects parameters from (1), (2) and (3). The parameters contained in θ are to be trained from data. The hyperparameters of FNO include the channel width d_c , the Fourier cut-off $k_{\max}$, the depth L and additional hyperparameters specifying the input and output layers $\mathcal{R}$ and $\mathcal{Q}$.

In theory, the FNO is formulated directly on function space and does not involve a reduction to a finite-dimensional latent space. In a practical implementation, it is usually discretized by identifying the input and output functions with their point-values on an equidistant grid. In this case, the discrete Fourier transform can be conveniently evaluated using the fast Fourier transform algorithm (FFT), and straightforward implementation in popular deep learning libraries is possible.

3.4. Random Features Method. The operator learning architectures above are usually trained from data using stochastic gradient descent. Whilst this shows great empirical success, the inability to analyze the optimization algorithms used by practitioners makes it difficult to make definitive statements about the networks that are trained in practice. The authors of [101] have proposed a randomized alternative, by extending the random features methodology [114] to a function space setting; this methodology has the advantage of being trainable through solution of a quadratic optimization problem.

The random feature model (RFM) requires specification of a parametrized operator $\psi : \mathcal{U} \times \Gamma \rightarrow \mathcal{V}$ with parameter set Γ , and a probability measure ν on the

parameters Γ . Each draw $\gamma \sim \nu$ specifies a random feature $\psi(\cdot; \gamma) : \mathcal{U} \rightarrow \mathcal{V}$, i.e. a random operator. Given iid samples $\gamma_1, \dots, \gamma_M \sim \nu$, the RFM operator is then defined as

$$\Psi_{RFM}(u; \theta)(y) = \sum_{j=1}^M \theta_j \psi(u; \gamma_j)(y) \quad \forall u \in \mathcal{U}, y \in \mathfrak{D}_y; \quad \gamma_j \text{ i.i.d..}$$

Here $\theta_1, \dots, \theta_M$ are scalar parameters. In contrast to the methodologies outlined above, the random feature model keeps the randomly drawn parameters $\gamma_1, \dots, \gamma_M$ fixed, and only optimizes over the coefficient vector $\theta = (\theta_1, \dots, \theta_M)$. With conventional loss functions, the resulting optimization over θ is convex, allowing for efficient and accurate optimization and a unique minimizer to be determined.

A suitable choice of random features is likely problem-dependent. Among others, DeepONet and FNO with randomly initialized weights are possible options. In the original work [101], the authors employ Fourier space random features (RF) for their numerical experiments, resembling a single-layer FNO. These Fourier space RF are specified by $\psi(u; \gamma) = \sigma(\mathcal{F}^{-1}(\chi \mathcal{F} \gamma \mathcal{F} u))$, where $\mathcal{F}$ denotes Fourier transform, σ an activation function, and χ is a Fourier space reshuffle, and γ is drawn from a Gaussian random field.

3.5. Discussion. The architectures above can be roughly divided into two categories, depending on how the underlying ideas from deep learning are leveraged to define a parametrized class of mappings on function space.

Encoder-Decoder Network Structure. The first approach, which we refer to as encoder-decoder-net and which includes the PCA-Net and DeepONet architectures, involves three steps: first, the input function is encoded by a finite-dimensional vector; second, an ordinary neural network, such as a fully connected or convolutional neural network, is employed to map the encoded input to a finite-dimensional output; third, a decoder maps the finite-dimensional output to an output function in the infinite-dimensional function space.

This approach is very natural from a numerical analysis point of view, sharing the basic structure of many numerical schemes, such as finite element methods (FEM), finite volume methods (FVM), and finite difference methods (FDM), as illustrated in Table 1. From this point of view, encoder-decoder-nets mainly differ from standard numerical schemes by replacing the hand-crafted algorithm and choice of numerical discretization by a data-driven algorithm encoded in the weights and biases of a neural network, and the possibility for a data-driven encoding and reconstruction. While appealing, such structure yields approximations within a fixed, finite-dimensional, linear subspace of $\mathcal{V}$. In particular, each output function from the approximate operator belongs to this linear subspace independently of the input function. Therefore these methods fall within the category of linear approximation, while methods for which outputs lie on a nonlinear manifold in $\mathcal{V}$ lead to what is known as *nonlinear approximation*. The benefits of nonlinear approximation are well understood in the context of functions [38], however, for the case of operators, results are still sparse but benefits for some specific cases have been observed [27, 69, 75, 79]. The FNO [83] and random features [101] are concrete examples of operator learning methodologies for which the outputs lie on a nonlinear manifold in $\mathcal{V}$.

Neural Operators Generalizing Neural Network Structure. A second approach to defining a parametrized class of operators on function space, distinct from encoder-decoder-nets, is illustrated by FNO. Following this approach, the structure of neural networks, which consist of an alternating composition of affine and nonlinear layers, is retained and generalized to function space. Nonlinearity is introduced

Method	Encoding	Finite-dim. Mapping	Reconstruction
FEM	Galerkin projection	Numerical scheme	Finite element basis
FVM	Cell averages	Numerical scheme	Piecewise polynomial
FD	Point values	Numerical scheme	Interpolation
PCA-Net	PCA projection	Neural network	PCA basis
DeepONet	Linear encoder	Neural network	Neural network basis

TABLE 1. Numerical schemes vs. Encoder-Decoder-Net.

via composition with a standard activation function, such as rectified linear unit or smooth variants thereof. The affine layers are obtained by integrating the input function against an integral kernel; this introduces nonlocality which is clearly needed if the architecture is to benefit from universal approximation.

Optimization and Randomization. The random features method [101] can in principle be combined with any operator learning architecture. The random features approach opens up a less explored direction of combining optimization with randomization in operator learning. In contrast to optimization of all parameters (by gradient descent) within a neural operator approach, the RFM allows for in-depth analysis resulting in error and convergence guarantees that take into account the finite number of samples, the finite number of parameters and the optimization [76]. One interesting, and largely unresolved question is how to design efficient random features for the operator learning setting.

The RFM is closely related to kernel methods which have a long pedigree in machine learning. In this context, we mention a related kernel-based approach proposed in [12], which employs kernel methods for operator learning within the encoder-decoder-net paradigm. This approach has shown to be competitive on several benchmark operator learning problems, and has been analyzed in [12].

Other Approaches. This review is mostly focused on methods that fall into one of the neural network-based approaches above, but it should be emphasized that other approaches are being actively pursued with success. Without aiming to present an exhaustive list, we mention nonlinear reduced-order modeling [79, 87, 111, 112, 134], approaches based on the theory of Koopman operators [81, 99, 108, 145], work aiming to augment and speed up numerical solvers [131], or work on data-driven closure modeling [58, 91, 139–141], to name just a few examples. For a broader overview of other approaches to machine learning for PDEs, we refer to the recent review [20]. While most “operator learning” is focused on operators mapping between functions with spatial or spatio-temporal dependence and often arising in connection with PDEs, we note that problems involving time-series represent another important avenue of machine learning research, which can also be viewed from, and may benefit from, the continuous viewpoint [77].

4. UNIVERSAL APPROXIMATION

The goal of the methodologies summarized in the last section is to approximate operators mapping between infinite-dimensional Banach spaces of functions. The first theoretical question to be addressed is whether these methods can achieve this task, even in principle? The goal of this line of research is to identify classes of operators for which operator learning methodologies possess a universal approximation property, i.e. the ability to approximate a wide class of operators to any given accuracy in the absence of any constraints on the model size, the number of data samples and without any constraints on the optimization.

Universal approximation theorems are well-known for finite dimensional neural networks mapping between Euclidean spaces [34, 55], providing a theoretical underpinning for their use in diverse applications. These results show that neural networks with non-polynomial activation can approximate very general classes of continuous (and even measurable) functions to any degree of accuracy. Universal approximation theorems for operator learning architectures provide similar guarantees in the infinite-dimensional context.

4.1. Encoder-Decoder-Nets. As pointed out in the last section, a popular type of architecture follows the encoder-decoder-net paradigm. Examples include PCA-Net and DeepONet. The theoretical basis for operator learning broadly, and within this paradigm more specifically, was laid out in a paper by Chen and Chen [24] in 1995, only a few years after the above cited results on the universality of neural networks. In that work, the authors introduce a generalization of neural networks, called operator networks, and prove that the proposed architecture possesses a universal property: it is shown that (shallow) operator networks can approximate, to arbitrary accuracy, continuous operators mapping between spaces of continuous functions. This architecture and analysis forms the basis of DeepONet, where the shallow neural networks of the original architecture of [24] are replaced by their deep counterparts. We present first a general, abstract version of an encoder-decoder-net and give a criterion on the spaces $\mathcal{U}, \mathcal{V}$ for which such architectures satisfy universal approximation. We then summarize specific results for DeepONet and PCA-Net architectures.

We call an encoder-decoder-net a mapping $\Psi_{ED} : \mathcal{U} \rightarrow \mathcal{V}$ which has the form

$$\Psi_{ED} = F_{\mathcal{U}} \circ \alpha \circ G_{\mathcal{V}}$$

where $F_{\mathcal{U}} : \mathcal{U} \rightarrow \mathbb{R}^{d_{\mathcal{U}}}$, $G_{\mathcal{V}} : \mathbb{R}^{d_{\mathcal{V}}} \rightarrow \mathcal{V}$ are bounded, linear maps and $\alpha : \mathbb{R}^{d_{\mathcal{U}}} \rightarrow \mathbb{R}^{d_{\mathcal{V}}}$ is a continuous function. The following theorem [68, Lemma 22] asserts that encoder-decoder-nets satisfy universal approximation over a large class of spaces $\mathcal{U}$ and $\mathcal{V}$.

Theorem 4.1. Suppose that $\mathcal{U}, \mathcal{V}$ are separable Banach spaces with the approximation property. Let $\Psi^{\dagger} : \mathcal{U} \rightarrow \mathcal{V}$ be a continuous operator. Fix a compact set $K \subset \mathcal{U}$. Then for any $\epsilon > 0$, there exist positive integers $d_{\mathcal{U}}, d_{\mathcal{V}}$, bounded linear maps $F_{\mathcal{U}} : \mathcal{U} \rightarrow \mathbb{R}^{d_{\mathcal{U}}}$, $G_{\mathcal{V}} : \mathbb{R}^{d_{\mathcal{V}}} \rightarrow \mathcal{V}$, and a function $\alpha \in C(\mathbb{R}^{d_{\mathcal{U}}}; \mathbb{R}^{d_{\mathcal{V}}})$, such that

$$\sup_{u \in K} \|\Psi^{\dagger}(u) - (F_{\mathcal{U}} \circ \alpha \circ G_{\mathcal{V}})(u)\|_{\mathcal{V}} \leq \epsilon.$$

◇

A Banach space is said to have the *approximation property* if, over any compact set, the identity map can be resolved as the limit of finite rank operators [86]. Although it is a fundamental property useful in many areas in approximation theory, it is not satisfied by all separable Banach spaces [43]. However, many of the Banach spaces used in PDE theory and numerical analysis such as Lebesgue spaces, Sobolev spaces, Besov spaces, and spaces of continuously differentiable functions all possess the approximation property [68, Lemma 26]. The above therefore covers a large range of scenarios in which encoder-decoder-nets can be used. We now give examples of encoder-decoder-nets where we fix the exact functional form of $F_{\mathcal{U}}$ and $G_{\mathcal{V}}$ and show that universal approximation continues to hold.

4.1.1. Operator Network and DeepONet. The specific form of operator networks as introduced and analysed by Chen and Chen in [24] focuses on scalar-valued input and output functions. We will state the main result of [24] in this setting for

notational simplicity. Extension to vector-valued functions is straight-forward. In simplified form, Chen and Chen [24, cf. Theorem 5] obtain the following result:

Theorem 4.2. Suppose that $\sigma \in C(\mathbb{R})$ is a non-polynomial activation function. Let $\mathfrak{D} \subset \mathbb{R}^d$ be a compact domain with Lipschitz boundary. Let $\Psi^\dagger : C(\mathfrak{D}) \rightarrow C(\mathfrak{D})$ be a continuous operator. Fix a compact set $K \subset C(\mathfrak{D})$. Then for any $\varepsilon > 0$, there are positive integers d_U, d_V, N , sensor points $x_1, \dots, x_{d_U} \in \mathfrak{D}$, and coefficients $c_i^k, \xi_{ij}^k, b_i, \omega_k, \zeta_k$ with $i = 1, \dots, N, j = 1, \dots, d_U, k = 1, \dots, d_V$, such that

$$(4.1) \quad \sup_{u \in K} \sup_{x \in \mathfrak{D}} \left| \Psi^\dagger(u)(x) - \sum_{k=1}^{d_V} \sum_{i=1}^N c_i^k \sigma \left(\sum_{j=1}^{d_U} \xi_{ij}^k u(x_j) + b_i \right) \sigma(\omega_k x + \zeta_k) \right| \leq \varepsilon.$$

◇

Here, we can identify the linear encoder $Lu = (u(x_1), \dots, u(x_{d_U}))$, the shallow branch-net α , with components

$$\alpha_k(Lu) = \sum_{i=1}^N c_i^k \sigma \left(\sum_{j=1}^{d_U} \xi_{ij}^k u(x_j) + b_i \right),$$

and trunk-net ψ , with components

$$\psi_k(y) = \sigma(\omega_k y + \zeta_k).$$

With these definitions, (4.1) can be written in the equivalent form,

$$\sup_{u \in K} \left\| \Psi^\dagger(u) - \sum_{k=1}^{d_V} \alpha_k(Lu) \psi_k \right\|_{C(\mathfrak{D})} \leq \varepsilon.$$

Remark 4.3. Theorem 4.2 holds in much greater generality. In particular, it is not necessary to consider operator mapping input functions to output functions on the same domain $\mathfrak{D}$. In fact, the same result can be obtained for operators $\Psi^\dagger : C(V) \rightarrow C(\mathfrak{D})$, where the input “functions” $u \in C(V)$ can have domain a compact subset V of a general, potentially infinite-dimensional, Banach space. ◇

Theorem 4.2 provides the motivation and theoretical underpinning for DeepONet, extended to deep branch- and trunk-nets in [92]. These results demonstrate the universality of DeepONet for a very wide range of operators, with approximation error measured in the supremum-norm over a compact set of input functions.

Related Work. Several extensions and variants of DeepONets have been proposed after the initial work by Lu *et al.* [92], including extensions of the universal approximation analysis. We provide a short overview of relevant works that include a theoretical component below.

Input Functions Drawn From a Probability Measure. In [74], Theorem 4.2 has been generalized to input functions drawn from a general input measure μ , including the case of unbounded support, such as a Gaussian measure. The error is correspondingly measured in the Bochner $L^2(\mu)$ -norm (cp. discussion of PCA-Net universality below), and it is demonstrated that DeepONet can approximate general Borel measurable operators in such a setting.

Alternative Encoders. There is work addressing the discretization-invariance of the encoding in DeepONet, resulting in architectures that allow for encoding of the input function at arbitrary sensor locations include Bel(Basis enhanced learning)-Net [149] and VIDON (Variable-input deep operator networks) [110].

The authors of [56] introduce Basis Operator Network, a variant of DeepONet, where encoding is achieved by projection onto a neural network basis. Universal approximation results are obtained, including encoding error estimates for this approach.

In [60], the authors address the issue of multiple input functions, and propose MIO-Net (Multiple Input/Output Net), based on tensor-product representations. The authors prove a universal approximation property for the resulting architecture, and demonstrate its viability in numerical experiments.

DeepONets on Abstract Hilbert Spaces. DeepONets mapping between abstract Hilbert spaces have been considered in [21, 22], including a discussion of their universality in that context.

4.1.2. PCA-Net. At a theoretical level, PCA-Net shares several similarities with DeepONet and much of the analysis can be carried out along similar lines. In addition to proposing the PCA-Net architecture and demonstrating its viability on numerical test problems including the Darcy flow and viscous Burgers equations, the authors of [14] also prove that PCA-Net is universal for operators mapping between infinite-dimensional Hilbert spaces, with approximation error measured in the Bochner $L^2(\mu)$ -norm with respect to the input measure μ . This initial analysis was developed and sharpened considerably in [72]; as an example of this we quote [72, Proposition 31].

Theorem 4.4 (PCA-Net universality). Let $\mathcal{U}$ and $\mathcal{V}$ be separable Hilbert spaces and let $\mu \in \mathcal{P}(\mathcal{U})$ be a probability measure on $\mathcal{U}$. Let $\Psi^\dagger : \mathcal{U} \rightarrow \mathcal{V}$ be a μ -measurable mapping. Assume the following moment conditions,

$$\mathbb{E}_{u \sim \mu}[\|u\|_{\mathcal{U}}^2], \mathbb{E}_{u \sim \mu}[\|\Psi^\dagger(u)\|_{\mathcal{V}}^2] < \infty.$$

Then for any $\varepsilon > 0$, there are dimensions $d_{\mathcal{U}}, d_{\mathcal{V}}$, a requisite amount of data N , a neural network ψ depending on this data, such that the PCA-Net, $\Psi = G_{\mathcal{V}} \circ \psi \circ F_{\mathcal{U}}$, satisfies

$$\mathbb{E}_{\{u_j\} \sim \mu^{\otimes N}} [\mathbb{E}_{u \sim \mu} [\|\Psi^\dagger(u) - \Psi(u; \{u_j\}_{j=1}^N)\|_{\mathcal{V}}^2]] < \varepsilon,$$

where the outer expectation is with respect to the iid data samples $u_1, \dots, u_N \sim \mu$, which determine the empirical PCA encoder and reconstruction. $\diamond$

4.2. Neural Operators. The Fourier neural operator (FNO) is a specific instance of a more general notion of neural operators [68]. The general structure of such neural operators is identical to that of the FNO, i.e. a composition

$$(4.2) \quad \begin{aligned} \Psi_{NO}(u; \theta) &= \mathcal{Q} \circ \mathcal{L}_L \circ \dots \circ \mathcal{L}_2 \circ \mathcal{L}_1 \circ \mathcal{R}(\mathcal{U}), \quad \forall u \in \mathcal{U}, \\ \mathcal{L}_\ell(v)(x; \theta) &= \sigma(W_\ell v(x) + b_\ell + \mathcal{K}(v)(x; \gamma_\ell)), \end{aligned}$$

except that the convolutional operator in each layer is replaced by a more general integral operator,

$$\mathcal{K}(v)(x; \gamma) = \int_{\mathcal{D}} \kappa(x, y; \gamma) v(y) dy.$$

Here, the integral kernel $\kappa(x, y; \gamma)$ is a matrix-valued function of x and y , parametrized by γ . Additional nonlinear dependency on the input function is possible yielding, for example, $\kappa = \kappa(x, y, u(x), u(y); \gamma)$; this structure is present in transformer models [136]. Different concrete implementations of such neural operators mostly differ in their choice of the parametrized integral kernel. For example, [83] uses Fourier basis, [135] uses wavelet basis, and [16] uses spherical harmonics. Other approaches restrict the support of κ [82] or assume it decays quickly away from its diagonal [71, 84]. For a more thorough review of this methodology, we refer to [68].

The universality of the FNO has first been established in [67], using ideas from Fourier analysis and, in particular, building on the density of Fourier series to show

that FNO can approximate a wide variety of operators. Given the great variety of possible alternative neural operator architectures, which differ from the FNO essentially only in their choice of the parametrized kernel, a proof of universality that does not explicitly rely on Fourier analysis, and which applies to a wide range of choice for the integral kernel is desirable. This has been accomplished in [73], where the authors remove from the FNO all non-essential components, from the perspective of universal approximation, yielding a minimal architecture termed the “averaging neural operator” (ANO).

4.2.1. Averaging Neural Operator. Up to non-essential details, the ANO introduced in [73] is a composition of nonlinear layers of the form,

$$\mathcal{L}(v; \gamma_\ell)(x) = \sigma \left(W_\ell v(x) + b_\ell(x) + V_\ell \int_{\mathfrak{D}} v(y) dy \right), \quad (\ell = 1, \dots, L),$$

where $W_\ell, V_\ell \in \mathbb{R}^{d_c \times d_c}$ are matrices, and $b_\ell(x)$ is a bias function. In the present work, to parametrize the bias functions, we consider bias of the form $b_\ell(x) = A_\ell x + c_\ell$ for matrix $A_\ell \in \mathbb{R}^{d_c \times d}$ and bias vector $c_\ell \in \mathbb{R}^{d_c}$. With this choice, the nonlinear layer $v \mapsto \mathcal{L}(v)$ takes the form,

$$\mathcal{L}(v; \gamma_\ell)(x) = \sigma \left(W_\ell v(x) + A_\ell x + c_\ell + V_\ell \int_{\mathfrak{D}} v(y) dy \right).$$

The parameter $\gamma_\ell = \{W_\ell, V_\ell, A_\ell, c_\ell\}$ collects the tunable parameters of the ℓ -th layer. To define an operator $\Psi : \mathcal{U}(D; \mathbb{R}^{d_i}) \rightarrow \mathcal{V}(D; \mathbb{R}^{d_o})$, we combine these nonlinear layers with linear input and output layers $\mathcal{R} : u(x) \mapsto Ru(x)$ and $\mathcal{Q} : v(x) \mapsto Qv(x)$, obtained by multiplication with matrices $R \in \mathbb{R}^{d_c \times d_i}$ and $Q \in \mathbb{R}^{d_o \times d_c}$, respectively. The resulting ANO is an operator of the form,

$$\Psi(u; \theta) = \mathcal{Q} \circ \mathcal{L}_L \circ \dots \circ \mathcal{L}_1 \circ \mathcal{R}(u),$$

with θ collecting the tunable parameters in each hidden layer, and the input and output layers.

The ANO can be thought of as a special case of FNO, where the convolutional integral kernel is constant, leading to the last term in each hidden layer being an integral or “average” of the input function. Similarly, the ANO is a special case of many other parametrizations of the integral kernel in neural operator architectures. Despite its simplicity, the ANO can nevertheless be shown to have a universal approximation property. We here cite a special case for operator mapping between continuous functions, and refer to [73] for more general results:

Theorem 4.5. Suppose that $\sigma \in C(\mathbb{R})$ is a non-polynomial activation function. Let $\mathfrak{D} \subset \mathbb{R}^d$ be a compact domain with Lipschitz boundary. Let $\Psi^\dagger : C(\mathfrak{D}) \rightarrow C(\mathfrak{D})$ be a continuous operator. Fix a compact set $K \subset C(\mathfrak{D})$. Then for any $\varepsilon > 0$, there exists an ANO $\Psi : C(\mathfrak{D}) \rightarrow C(\mathfrak{D})$ such that

$$\sup_{u \in K} \|\Psi^\dagger(u) - \Psi(u)\|_{C(\mathfrak{D})} < \varepsilon.$$

◇

As an immediate consequence, we have the following corollary which implies universality of neural operators for a wide range of choices:

Corollary 4.6. Consider any neural operator architecture of the form (4.2) with parametrized integral kernel $\kappa(x, y; \gamma)$. If for any channel dimension d_c and matrix $V \in \mathbb{R}^{d_c \times d_c}$, there exists a parameter γ_V such that $\kappa(x, y; \gamma_V) \equiv V$, then the neural operator architecture is universal in the sense of Theorem 4.5.

◇

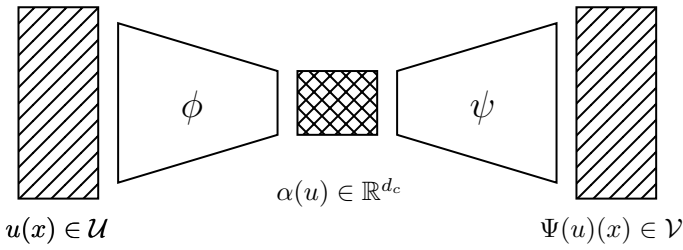


FIGURE 5. Special case of an averaging neural operator, illustrated as a nonlinear encoder-decoder architecture; with encoder $u \mapsto \alpha = \int_{\mathfrak{D}} \phi(u(y), y) dy$, and decoder $\alpha \mapsto \psi(\alpha, \cdot)$.

The last corollary applies in particular to the FNO. Universality of the FNO was first established in [67], there restricting attention to a periodic setting but allowing for operators mapping between general Sobolev spaces. The idea of the averaging neural operator can be found in [73], where it was used to prove universality for a wide range of neural operator architectures, and for a class of operators mapping between general Sobolev spaces, or spaces of continuously differentiable functions.

Intuition. We would like to provide further intuition for the universality of ANO. A simple special case of the ANO is the two-layer ANO obtained as follows. We consider neural operators which can be written as a composition of two shallow neural networks $\phi : \mathbb{R}^{d_i} \times \mathfrak{D} \rightarrow \mathbb{R}^{d_c}$, $\psi : \mathbb{R}^{d_c} \times \mathfrak{D} \rightarrow \mathbb{R}^{d_o}$ and an additional integral (average):

$$\begin{cases} \alpha(u) = \int_{\mathfrak{D}} \phi(u(y), y) dy, \\ \Psi(u)(x) = \psi(\alpha(u), x), \end{cases}$$

where

$$\phi(w, x) = C_1 \sigma(A_1 w + B_1 x + d_1),$$

and

$$\psi(z, x) = C_2 \sigma(A_2 z + B_2 x + d_2).$$

Note that the above composition, i.e.

$$\Psi(u)(x) = \psi \left(\int_{\mathfrak{D}} \phi(u(y), y) dy, x \right),$$

indeed defines a special case of ANO that can be written as a composition of a trivial input layer, two hidden layers and an output layer:

$$\begin{aligned} \mathcal{L}_1(u)(x) &= \sigma(W_1 u(x) + b_1(x)), & b_1(x) &= B_1 x + d_1, W_1 = A_1, \\ \mathcal{L}_2(v)(x) &= \sigma \left(b_2(x) + V_2 \int v(y) dy \right), & b_2(x) &= B_2 x + d_2, V_2 = A_2 C_1 \\ \mathcal{Q}(v)(x) &= Q v(x), & Q &= C_2. \end{aligned}$$

As depicted in Figure 5, such shallow ANO can be interpreted as a composition of an nonlinear encoder $\alpha : \mathcal{U} \rightarrow \mathbb{R}^{d_c}$, $u \mapsto \alpha(u)$ defined via spatial averaging of ϕ , and mapping the input function to a finite-dimensional latent space, and a nonlinear decoder $\psi : \mathbb{R}^{d_c} \rightarrow \mathcal{V}$, $\alpha \mapsto \psi(\alpha, \cdot)$. This interpretation opens up a path for analysis, based on which universality can be established for the ANO, and any neural operator architecture that contains such ANO as a special case, such as the Fourier neural operator.

The theoretical contributions outlined in the previous section are mostly focused on methodological advances and a discussion of the universality of the resulting architectures. Universality of neural operator architectures, i.e. the ability to approximate a wide class of operators, is arguably a necessary condition for their success. But since universality is inherently qualitative, it cannot explain the efficiency of these methods in practice. Improving our understanding of the efficiency of neural operators in practice requires a quantitative theory of operator learning, providing explicit error and complexity estimates: given a desired accuracy $\varepsilon > 0$, what is the model size or the number of samples that is required to achieve such accuracy?

5.1. Linear Operators. Learning a linear operator can be formulated as solving a linear inverse problem with a non-compact forward operator [36, 98]. We take this point of view and broadly describe the results from [36]. We consider the problem of learning $\Psi^\dagger = L^\dagger$, a linear operator, when $\mathcal{U} = \mathcal{V}$ is a separable Hilbert space. Studying the linear problem enables a thorough analysis of the complexity of operator learning and hence sheds light on the problem in the more general setting. The work proceeds by assuming that $L^\dagger$ can be diagonalized in a known Schauder basis of $\mathcal{U}$ denoted $\{\varphi_j\}_{j=1}^\infty$. Given input data $\{u_n\}_{n=1}^N \stackrel{\text{i.i.d.}}{\sim} \mu$, the noisy observations are assumed to be of the form

$$v_n = L^\dagger u_n + \gamma \xi_n$$

where $\gamma > 0$ and the sequence $\{\xi_n\}_{n=1}^N \stackrel{\text{i.i.d.}}{\sim} \mathcal{N}(0, I_{\mathcal{U}})$ comes from a Gaussian white noise process that is independent of the input data $\{u_n\}$. In what follows here, we assume that $\mu = \mathcal{N}(0, \Gamma)$, the measure on the input data, is Gaussian with a strictly positive covariance Γ and that this covariance is also diagonalizable by $\{\varphi_j\}$; note, however, that [36] treats the more general case without assuming simultaneous diagonalizability of $L^\dagger$ and Γ .

Note that $\{l_j^\dagger, \varphi_j\}$ uniquely determines $L^\dagger$. Thus the problem as formulated here can be stated as learning the eigenvalue sequence $\{l_j^\dagger\}_{j=1}^\infty$ of $L^\dagger$ from the noisy observations

$$v_{jn} = \langle \varphi_j, u_n \rangle_{\mathcal{U}} l_j^\dagger + \xi_{jn}, \quad j \in \mathbb{N}, \quad n = 1, \dots, N$$

where $\{\xi_{jn}\} \stackrel{\text{i.i.d.}}{\sim} \mathcal{N}(0, \gamma^2)$. In this problem statement, the noise is crucial for obtaining meaningful estimates on the amount of data needed for learning. Indeed, without noise, the eigenvalues can simply be recovered as

$$l_j^\dagger = \frac{v_{j1}}{\langle \varphi_j, u_1 \rangle_{\mathcal{U}}}$$

for all $j \in \mathbb{N}$ from a single data point u_1 since the basis $\{\varphi_j\}$ is assumed to be known. While, in practice, the observations might not be noisy, the noise process can be used to model round-off or discretization errors which occur in computation.

Assuming a Gaussian prior on the sequence $\{l_j^\dagger\} \sim \otimes_{j=1}^\infty \mathcal{N}(0, \sigma_j^2)$, one may obtain a Bayesian estimator of $L^\dagger$, given the data $(\{v_{jn}\}, \{u_n\})$. The Bayesian posterior is characterized as an infinite Gaussian product for the sequence of eigenvalues. We take as an estimator the mean of this Gaussian which is given as

$$l_j = \frac{\gamma^{-2} \sigma_j^2 \sum_{n=1}^N v_{jn} \langle \varphi_j, u_n \rangle_{\mathcal{U}}}{1 + \gamma^{-2} \sigma_j^2 \sum_{n=1}^N |\langle \varphi_j, u_n \rangle_{\mathcal{U}}|^2}.$$

Then our estimator is the operator Ψ , diagonalized in basis $\{\varphi_j\}$ with eigenvalues $\{l_j\}$. To quantify the smoothness of $L^\dagger$, we assume that $\{l_j^\dagger\}$ lives in an appropriately

weighted ℓ^2 space, in particular,

$$\sum_{j=1}^{\infty} j^{2s} |l_j^\dagger|^2 < \infty$$

for some $s \in \mathbb{R}$. Then the following theorem holds [36, Theorem 1.3].

Theorem 5.1. Suppose that for some $\alpha > 1/2$ and $p \in \mathbb{R}$, we have $\langle \varphi_j, \Gamma \varphi_j \rangle_{\mathcal{U}} \asymp j^{-2\alpha}$ and $\sigma_j^2 \asymp j^{-2p}$. Let $\alpha' \in [0, \alpha + 1/2)$ and assume that $\min\{\alpha, \alpha'\} + \min\{p - 1/2, s\} > 0$. Then, as $N \rightarrow \infty$, we have

$$\mathbb{E} \sum_{j=1}^{\infty} j^{-2\alpha'} |l_j - l_j^\dagger|^2 \lesssim N^{-\frac{\alpha' + \min\{p-1/2, s\}}{\alpha+p}}$$

where the expectation is taken over the product measure defining the input data and noise. $\diamond$

Theorem 5.1 quantifies the amount of data needed, on average, for the estimator $\{l_j\}$ to achieve ϵ -error in approximating $\{l_j^\dagger\}$ measured in a squared weighted ℓ^2 norm. In particular, we have

$$N \sim \epsilon^{-\frac{\alpha+p}{\alpha' + \min\{p-1/2, s\}}}.$$

The exact dependence of this rate on the parameters defining the smoothness of the truth, the input, the prior, and the error metric elucidates the optimal design choices for the estimator and sheds light on which pieces are most important for the learning process. We refer to [36] for an in-depth discussion.

Within machine learning and functional data analysis, many works have focused on learning integral kernel operators [1, 33, 59, 119, 138]. The operator learning problem can then be reduced to approximation of the kernel function and is typically studied in a Reproducing Kernel Hilbert Space setting. In numerical PDEs, some recent works have studied the problem of learning the Green's function arising from some elliptic, parabolic and hyperbolic problems [17–19, 137].

5.2. Holomorphic Operators. Going beyond the linear case, holomorphic operators represent a very general class of operators for which *efficient* quantitative error and complexity estimates can be established. We mention the influential work by Cohen, DeVore and Schwab [29, 30], as well as further developments [3–7, 25, 26, 52, 94, 103, 124, 125]. A detailed review, from 2015, can be found in [28]. We will only describe a few main ideas. We mention in passing also the works [70, 80], which study the neural network approximation of parametric operators in a related setting.

Holomorphic operators have mostly been studied in a parametrized setting, where the input functions can be identified with the coefficients in a suitable basis (frame) expansion. A prototypical example is the elliptic Darcy flow equation (2.5), where the coefficient field $a = a(\cdot; \mathbf{y})$ is parametrized by a sequence $\mathbf{y} = (y_1, y_2, \dots) \in [-1, 1]^{\mathbb{N}}$, e.g. in the form of a linear expansion,

$$a(x; \mathbf{y}) = \bar{a}(x) + \sum_{j=1}^{\infty} y_j \varphi_j(x), \quad x \in \mathfrak{D},$$

where $\bar{a} \in \mathcal{U}$ and $\varphi_1, \varphi_2, \dots \in \mathcal{U}$ are fixed. The operator $\Psi^\dagger : a \mapsto u$ can then (loosely) be identified with the parametrized mapping,

$$\mathbf{F} : [-1, 1]^{\mathbb{N}} \rightarrow \mathcal{V}, \quad \mathbf{F}(\mathbf{y}) := \Psi^\dagger(a(\cdot; \mathbf{y})).$$

In the above prototypical setting, and assuming that the sequence $\mathbf{b}$ with coefficients $b_j = \|\varphi_j\|_{\mathcal{V}}$ decays sufficiently fast, it can be shown [28] that F possesses a holomorphic extension to a subset of the infinite product $\mathbb{C}^{\mathbb{N}} = \prod_{j=1}^{\infty} \mathbb{C} = \mathbb{C} \times \mathbb{C} \times \dots$ of the complex plane $\mathbb{C}$. More precisely, there exists a holomorphic extension

$$(5.1) \quad F : \prod_{j=1}^{\infty} \mathcal{E}_{\rho_j} \rightarrow \mathcal{V}_{\mathbb{C}}, \quad \mathbf{z} \mapsto F(\mathbf{z}).$$

Here, for $\rho_j > 1$, the set $\mathcal{E}_{\rho_j} = \left\{ \frac{1}{2}(z + z^{-1}) \mid z \in \mathbb{C}, 1 \leq |z| \leq \rho_j \right\} \subset \mathbb{C}$ denotes the Bernstein ellipse with focal points ± 1 and major and minor semi-axes lengths $\frac{1}{2}(\rho_j \pm \rho_j^{-1})$, and $\mathcal{V}_{\mathbb{C}}$ is the complexification of the Banach space $\mathcal{V}$.

In general, given a non-negative sequence $\mathbf{b} \in [0, \infty)^{\mathbb{N}}$ and $\varepsilon > 0$, a parametrized operator $F : [-1, 1]^{\mathbb{N}} \rightarrow \mathcal{V}$ is called $(\mathbf{b}, \varepsilon)$ -holomorphic, if it possesses a holomorphic extension (5.1) for any $\boldsymbol{\rho} = (\rho_1, \rho_2, \dots) \in [1, \infty)^{\mathbb{N}}$, s.t.

$$(5.2) \quad \sum_{j=1}^{\infty} \left(\frac{\rho_j + \rho_j^{-1}}{2} - 1 \right) b_j \leq \varepsilon.$$

As reviewed in [5, Chapter 4], a number of parametric differential equations of practical interest give rise to $(\mathbf{b}, \varepsilon)$ -holomorphic operators.

The approximation theory of this class of operators is well-developed [5, 28]. The underlying reason why efficient approximation of such operators is possible is that holomorphic operators possess convergent expansions in multi-variate polynomial bases, where each polynomial basis element only depends on a finite number of the components of the complex input $\mathbf{z} = (z_1, z_2, \dots)$.

A standard setting considers $\mathbf{b} \in \ell^p(\mathbb{N})$, for some $0 < p < 1$. For example, if $b_j \sim j^{-s}$ decays algebraically, then $\mathbf{b} \in \ell^p(\mathbb{N})$ for $s > p^{-1} > 1$. Assuming that $\mathbf{b} \in \ell^p(\mathbb{N})$, it can be shown (e.g. [28, Corollary 3.11]) that the best n -term polynomial approximation to F converges at rate $n^{-1/p}$ in a sup-norm setting, and rate $n^{1/2-1/p}$ in a Bochner $L^2(\mu)$ -setting, with specific input measure μ on $[-1, 1]^{\mathbb{N}}$. Importantly, these convergence rates are polynomial in the number of degrees of freedom n , even in this infinite-dimensional parametric setting. When restricting to a finite-dimensional input space with d input components, i.e. considering only inputs of the form $\mathbf{z} = (z_1, \dots, z_d, 0, 0, \dots)$, this fact implies that convergence rates independent of the dimension d can be obtained, and thus such approximation of $(\mathbf{b}, \varepsilon)$ -holomorphic operators can provably overcome the *curse of dimensionality* [28].

The above mentioned results in the parametrized setting can also be used to prove efficient approximation of holomorphic operators by operator learning frameworks in a non-parametric setting [52, 74, 103, 124, 125]. For example, [52] consider the DeepONet approximation of holomorphic operators with general Riesz frame encoders and decoders, demonstrating algebraic error and complexity estimates; Under suitable conditions, the authors prove that ReLU deep operator networks (DeepONet) approximating holomorphic operators can achieve convergence rates arbitrarily close to n^{1-s} in a worst-error setting (supremum norm) and at rate $n^{1/2-s}$ in a Bochner $L^2(\mu)$ -norm setting. Here n denotes the number of tunable parameters of the considered DeepONet, and the parameter s determines the decay of the coefficients in the frame expansion of the considered input functions. Under the (loose) identification $s \sim 1/p$, these rates for DeepONet recover the rates discussed above. These results show that there exist operator surrogates which essentially achieve the approximation rates afforded by best n -term approximation schemes mentioned above.

The complementary question of the sample complexity of operator learning for holomorphic operators has been studied in [6, 7, 9]. Building on the theory of N -widths, the authors of [6, 7] show that on the class $(\mathbf{b}, \varepsilon)$ -holomorphic operators and in a Bochner L^2 -setting, data-driven methods relying on N samples cannot achieve convergence rates better than $N^{1/2-1/p}$. In addition, it is shown that the optimal rate can be achieved up to logarithmic terms. We refer to [6, 7] for further details.

To summarize: holomorphic operators represent a class of operators of practical interest for which approximation theory by neural operator learning frameworks can be developed. The approximation theory of this class of operators is well-developed, especially in the parametrized setting. In a parametrized setting, these operators allow for efficient approximation by multi-variate (sparse) polynomials. This fact can be leveraged to show that efficient approximation by neural network-based methods is possible, and such results can be extended beyond a parametric setting, e.g. via frame expansions. Optimal approximation rates, and methods achieving these optimal rates, up to logarithmic terms, are known under specific assumptions.

5.3. General (Lipschitz) Operators. The last two sections provide an overview of theoretical results on the approximation of linear and holomorphic operators. While these classes of operators include several operators of practical interest and allow for the development of general approximation theory, not all operators of relevance are holomorphic (or indeed linear). Examples of non-holomorphic operators include the solution operator associated with nonlinear hyperbolic conservation laws such as the compressible Euler equations. Solutions of such equations can develop shocks in finite time, and it can be shown that the associated solution operators themselves are not holomorphic. It is therefore of interest to extend the approximation theory of operator learning frameworks beyond the restrictive class of holomorphic operators.

A general and natural class of nonlinear operators are general Lipschitz continuous operators, the approximation theory of which has been considered from an operator learning perspective e.g. in [14, 45, 47, 90, 123]. We present a brief outline of the general approach and mention relevant work on model complexity estimates in the following subsections 5.3.1–5.3.3. Relevant results on the data complexity of operator learning in this setting are summarized in subsection 5.3.4.

5.3.1. Error Decomposition. Encoder-decoder-net architectures arguably follow the basic mathematical intuition of how to approach the operator approximation problem most closely, and most theoretical work has focused on this approach. We recall that within this framework, the infinite-dimensional input and output spaces $\mathcal{U}, \mathcal{V}$ are first encoded through suitable finite-dimensional latent spaces. This involves an encoder/decoder pair $(F_{\mathcal{U}}, G_{\mathcal{U}})$ on $\mathcal{U}$,

$$F_{\mathcal{U}} : \mathcal{U} \rightarrow \mathbb{R}^{d_{\mathcal{U}}}, \quad G_{\mathcal{U}} : \mathbb{R}^{d_{\mathcal{U}}} \rightarrow \mathcal{U},$$

and another encoder/decoder pair $(F_{\mathcal{V}}, g_{\mathcal{V}})$ on $\mathcal{V}$,

$$F_{\mathcal{V}} : \mathcal{V} \rightarrow \mathbb{R}^{d_{\mathcal{V}}}, \quad G_{\mathcal{V}} : \mathbb{R}^{d_{\mathcal{V}}} \rightarrow \mathcal{V}.$$

We recall that the composition $G_{\mathcal{U}} \circ F_{\mathcal{U}}, G_{\mathcal{V}} \circ F_{\mathcal{V}}$ are interpreted as approximations to the identity on $\mathcal{U}$ and $\mathcal{V}$, respectively. These encode/decoder pairs in turn imply an encoding of the underlying infinite-dimensional operator $\Psi^{\dagger} : \mathcal{U} \rightarrow \mathcal{V}$, resulting in a finite-dimensional function

$$\varphi : \mathbb{R}^{d_{\mathcal{U}}} \rightarrow \mathbb{R}^{d_{\mathcal{V}}}, \quad \varphi(\alpha) = F_{\mathcal{V}} \circ \Psi^{\dagger} \circ G_{\mathcal{U}}(\alpha),$$

as depicted earlier, in Figure 3.

While the encoder and decoder of these architectures perform dimension reduction, the neural network $\psi : \mathbb{R}^{d_u} \rightarrow \mathbb{R}^{d_v}$ at the core of encoder-decoder-net architectures is interpreted as approximating this resulting finite-dimensional function $\varphi : \mathbb{R}^{d_u} \rightarrow \mathbb{R}^{d_v}$. To summarize, an encoder-decoder-net can conceptually be interpreted as involving three steps:

- (1) Dimension reduction on the input space $\mathcal{U} \approx \mathbb{R}^{d_u}$,
- (2) Dimension reduction on the output space $\mathcal{V} \approx \mathbb{R}^{d_v}$,
- (3) Encoding of the operator $\Psi^\dagger$ yielding $\varphi : \mathbb{R}^{d_u} \rightarrow \mathbb{R}^{d_v}$, approximated by neural network $\psi : \mathbb{R}^{d_u} \rightarrow \mathbb{R}^{d_v}$.

Each part of this conceptual decomposition, the encoding of $\mathcal{U} \approx \mathbb{R}^{d_u}$, the decoding $\mathbb{R}^{d_v} \approx \mathcal{V}$ and the approximation $\psi \approx \varphi$, represents a source of error, and the total encoder-decoder-net approximation error $\mathcal{E}$ is bounded by three contributions $\mathcal{E} \lesssim \mathcal{E}_\mathcal{U} + \mathcal{E}_\psi + \mathcal{E}_\mathcal{V}$, where

$$\mathcal{E}_\mathcal{U} = \sup_u \|u - G_\mathcal{U} \circ F_\mathcal{U}(u)\|_\mathcal{U},$$

quantifies the encoding error, with supremum taken over the relevant set of input functions u ,

$$\mathcal{E}_\mathcal{V} = \sup_v \|v - G_\mathcal{V} \circ F_\mathcal{V}(v)\|_\mathcal{V},$$

quantifies the decoding error, and

$$(5.3) \quad \mathcal{E}_\psi = \sup_\alpha \|F_\mathcal{V} \circ \Psi^\dagger \circ G_\mathcal{U}(\alpha) - \psi(\alpha)\|,$$

is the neural network approximation error.

Given this decomposition, the derivation of error and complexity estimates for encoder-decoder-net architectures thus boils down to the estimation of encoding error $\mathcal{E}_\mathcal{U}$, neural network approximation error $\mathcal{E}_\psi$ and reconstruction error $\mathcal{E}_\mathcal{V}$, respectively.

Encoding and Reconstruction Errors. Encoding and reconstruction errors are relatively well understood on classical function spaces such as Sobolev and Besov spaces, by various linear and nonlinear methods of approximation [40].

For linear encoder/decoder pairs, the analysis of encoding and reconstruction errors amounts to principal component analysis (PCA) when measuring the error in the Bochner norm L^2_μ , or to Kolmogorov n-widths when measuring the error in the sup-norm over a compact set. Relevant discussion of PCA in the context of operator learning is given in [14] (see also [74] and [72]).

In certain settings, such as for PDEs with discontinuous output functions, it has been shown [75] that relying on linear reconstruction imposes fundamental limitations on the approximation accuracy of operator methodologies, which can be overcome by methods with nonlinear reconstruction; specifically, it was shown both theoretically and experimentally in [75] that FNO and shift-DeepONet, a variant of DeepONet with nonlinear reconstruction, achieve higher accuracy than vanilla DeepONet for prototypical PDEs with discontinuous solutions. We also mention closely related work on the nonlinear manifold decoder architecture of [126].

Neural Network Approximation Error. At their core, encoder-decoder-net architectures employ a neural network to approximate the encoded version $F_\mathcal{V} \circ \Psi^\dagger \circ G_\mathcal{U}$ of the underlying operator $\Psi^\dagger$ (cp. (5.3)). The practical success of these frameworks thus hinges on the ability of ordinary neural networks to approximate the relevant class of high-dimensional functions in the latent-dimensional spaces, which are obtained through the encoding of such operators. While the empirical success of neural networks in high-dimensional approximation tasks is undeniable, our theoretical understanding and the mathematical foundation underpinning this empirical success remains incomplete.

General approximation theoretic results on the neural network approximation of functions have been obtained, and some available quantitative bounds in operator learning [45,47] build on these results to estimate the neural network approximation error $\mathcal{E}_\psi$. Notably, the seminal work [142] of D. Yarotsky presents general error and complexity estimates for functions with Lipschitz continuous derivatives:

Theorem 5.2. A function $f \in W^{k,\infty}([0,1]^d)$ can be approximated to uniform accuracy $\varepsilon > 0$,

$$\sup_{x \in [0,1]^d} |f(x) - \psi(x)| \leq \varepsilon,$$

by a ReLU neural network ψ with at most $O(\varepsilon^{-d/k} \log(\varepsilon^{-1}))$ tunable parameters. $\diamond$

Remark 5.3. Note that the relevant dimension in the operator learning context is the latent dimension $d = d_{\mathcal{U}}$. Neglecting logarithmic terms, we note that each component of the function $G : \mathbb{R}^{d_{\mathcal{U}}} \rightarrow \mathbb{R}^{d_{\mathcal{V}}}$ can be approximated individually by a neural network of size at most $O(\varepsilon^{-d_{\mathcal{U}}/k})$, and hence, we expect that G can be approximated to accuracy ε by a neural network ψ of size at most $O(d_{\mathcal{V}} \varepsilon^{-d_{\mathcal{U}}/k})$. $\diamond$

Without aiming to provide a comprehensive overview of this very active research direction on neural network approximation theory, adjacent to operator learning theory, we mention that similar error and complexity estimates can also be obtained on more general Sobolev spaces, e.g. [130,144]. Lower bounds illuminating the limitations of neural networks on model classes are for example discussed in [2,15,142]. Approximation rates leveraging additional structure beyond smoothness have also been considered, e.g. compositional structure is explored in [96,122,128].

Non-standard Architectures and Hyperexpressive Activations. While the general research area of neural network approximation theory is too broad to adequately summarize here, we mention relevant work on hyperexpressive activations, which can formally break the curse of dimensionality observed Theorem 5.2; it has been shown that neural networks employing non-standard activations can formally achieve arbitrary convergence on model function classes [85,109,129,143], when the complexity is measured in terms of number of tunable parameters. This is not true for the ReLU activation [142]. Another way to break the curse of dimensionality is via architectures with non-standard “three-dimensional” structure [148].

While non-standard architectures can overcome the curse of dimensionality in the sense that the number of parameters does not grow exponentially with d (or is independent of d), it should be pointed out that this necessarily comes at the expense of the number of bits that are required to represent each parameter in a practical implementation. Indeed, from work on quantized neural networks [15] (with arbitrary activation function), it can be inferred that the total number of bits required to store all parameters in such architectures is lower bounded by the Kolmogorov ε -entropy of the underlying model class; For the specific model class $W^{k,\infty}([0,1]^d)$, this entropy scales as $\varepsilon^{-d/k}$. Hence, architectures which achieve error ε with a number of parameters that scales strictly slower than $\varepsilon^{-d/k}$ must do so at the expense of the precision that is required to represent each individual parameter in a practical implementation, keeping the total number of bits above the entropy limit. For related discussion, we e.g. refer to [144, section 7] or [130, discussion on page 5]. Another implication of this fact is that the constructed non-standard architectures are necessarily very sensitive to minute changes in the network parameters.

5.3.2. *Upper Complexity Bounds.* Quantitative error estimates for operator learning based on the general approach outlined in the last subsection 5.3.1 have been derived in a number of recent works [14, 45, 47, 56, 90, 95].

Relevant work. The two papers [14, 74], analyzing PCA-Net and DeepONet respectively, both introduce a splitting of the error into encoder, neural network approximation and reconstruction errors. A similar error analysis is employed in [56] for so-called “basis operator network”, a variant of DeepONet. An in-depth analysis of DeepONets with various encoder/decoder pairs, including generalization error estimates, is given in [90]. Quantitative approximation error estimates for convolution neural networks applied to operator learning are derived in [45]. General error estimates motivated by infinite-dimensional dynamical systems in stochastic analysis can be found in [47]. An alternative approach to operator learning with explicit algorithms for all weights is proposed in [95], including error estimates for this approach.

Alternative Decompositions. Finally, we point out that while the error decomposition in encoding, neural network approximation and reconstruction errors is natural, alternative error decompositions, potentially more fine-grained, are possible. We mention the work [105] which proposes a mimetic neural operator architecture inspired by the weak variational form of elliptic PDEs, discretized by the finite-element method; starting from this idea, the authors arrive at an architecture that can be viewed as a variant of DeepONet, including a specific mixed nonlinear and linear branch network structure and a nonlinear trunk net. In this work, an a priori error analysis is conducted resulting in a splitting of the overall approximation in numerical approximation, stability, training and quadrature errors depending on the data-generation with a numerical scheme (no access to the actual operator), the Lipschitz stability of the underlying operator, the finite number of training samples and quadrature errors to approximate integrals, respectively.

5.3.3. *Lower Complexity Bounds.* Operator learning frameworks are based on neural networks and provide highly nonlinear approximation [38]. Despite their astonishing approximation capabilities, even highly nonlinear approximation methods have intrinsic limitations.

Combined Error Analysis for Encoder-Decoder-Nets. To illustrate some of these intrinsic limitations, we first outline the combined error analysis that results from the decomposition summarized in the last section. To this end, we combine the encoding, reconstruction and neural network analysis to derive quantitative error and complexity estimates within the encoder-decoder-net paradigm.

Firstly, under reasonable assumptions on the input functions, the encoding error can often be shown to decay at an algebraic rate in the $d_{\mathcal{U}}$, e.g.

$$(5.4) \quad \mathcal{E}_{\mathcal{U}} \lesssim d_{\mathcal{U}}^{-\alpha}.$$

For example, if we assume that the input functions are defined on a bounded domain $\mathfrak{D} \subset \mathbb{R}^d$ and subject to a smoothness constraint such as a uniform bound on their k -th derivative, then a decay rate $\alpha = k/d$ can be achieved (depending on the precise setting); For dimension reduction by principal component analysis, the exponent α instead relates to the decay rate of the eigenvalues of the covariance operator of the input distribution.

Under similar assumptions on the set of output functions, depending on the properties of the underlying operator $\Psi^\dagger$, the reconstruction error on the output space often also decays algebraically,

$$(5.5) \quad \mathcal{E}_{\mathcal{V}} \lesssim d_{\mathcal{V}}^{-\beta},$$

where the decay rate β can e.g. be estimated in terms of the smoothness of the output functions under $\Psi^\dagger$, or could be related to the decay of the PCA eigenvalues of the output distribution (push-forward under $\Psi^\dagger$).

Finally, given latent dimensions $d_{\mathcal{U}}$ and $d_{\mathcal{V}}$, the size of the neural network ψ that is required to approximate the encoded operator mapping $G : \mathbb{R}^{d_{\mathcal{U}}} \rightarrow \mathbb{R}^{d_{\mathcal{V}}}$, with NN approximation error bound,

$$\mathcal{E}_\psi \leq \varepsilon,$$

roughly scales as (cp. Remark 5.3),

$$(5.6) \quad \text{size}(\psi) \sim d_{\mathcal{V}} \varepsilon^{-d_{\mathcal{U}}/k},$$

when the only information on the underlying operator is captured by its degree of smoothness k ($k = 1$ corresponding to Lipschitz continuity). Note that this is the scaling consistent with Kolmogorov entropy bounds.

Given the error decomposition $\mathcal{E} \lesssim \mathcal{E}_{\mathcal{U}} + \mathcal{E}_\psi + \mathcal{E}_{\mathcal{V}}$, we require each error contribution individually to be bounded by ε . In view of (5.4) and (5.5), this can be achieved provided that $d_{\mathcal{U}} \sim \varepsilon^{-1/\alpha}$, $d_{\mathcal{V}} \sim \varepsilon^{-1/\beta}$. Inserting such choice of $d_{\mathcal{U}}$, $d_{\mathcal{V}}$ in (5.6), we arrive at a neural network size of roughly the form,

$$\text{size}(\psi) \sim \varepsilon^{-1/\beta} \varepsilon^{-c\varepsilon^{-1/\alpha}/k}.$$

In particular, we note the exponential dependence on ε^{-1} , resulting in a size estimate,

$$(5.7) \quad \text{size}(\psi) \gtrsim \exp\left(\frac{c\varepsilon^{-1/\alpha}}{k}\right).$$

As pointed out after (5.4), when the set of input functions consists of functions defined on a d -dimensional domain with uniformly bounded s -th derivatives (in a suitable norm), then we expect a rate $\alpha = s/d$, in which case we obtain,

$$(5.8) \quad \text{size}(\psi) \gtrsim \exp\left(\frac{c\varepsilon^{-d/s}}{k}\right).$$

For operator learning frameworks, this super-algebraic (even exponential) dependence of the complexity on ε^{-1} has been termed the “curse of dimensionality” in [67, 74] or more recently “curse of parametric complexity” in [72]. The latter term was introduced to avoid confusion, which may arise because in these operator learning problems, there is no fixed dimension to speak of. The curse of parametric complexity can be viewed as an infinite-dimensional scaling limit of the finite-dimensional curse of dimensionality, represented by the $d_{\mathcal{U}}$ -dependency of the bound $\varepsilon^{-d_{\mathcal{U}}/k}$, and arises from the finite-dimensional CoD by observing that the required latent dimension $d_{\mathcal{U}}$ itself depends on ε , with scaling $d_{\mathcal{U}} \sim \varepsilon^{-1/\alpha}$. We note in passing that even if $d_{\mathcal{U}} \sim \log(\varepsilon^{-1})$ were to scale only logarithmically in ε^{-1} , the complexity bound implied by (5.6) would still be super-algebraic, consistent with the main result of [72].

Nonlinear n -width Estimates. The rather pessimistic complexity bound outlined in (5.7) is based on an upper bound on the operator approximation error $\mathcal{E}$, and is not necessarily tight. One may therefore wonder if more careful estimates could yield complexity bounds that do not scale exponentially in ε^{-1} .

In this context, we would like to highlight the early work on operator approximation by Mhaskar and Hahm [97] which presents first quantitative bounds for the approximation of nonlinear functionals; most notably, this work identifies the continuous nonlinear n -widths of spaces of Hölder continuous functionals defined on L^2 -spaces; it is shown that the relevant n -widths decay only (poly-)logarithmically in n , including both upper and lower bounds.

We will presently state a simplified version of the main result of [97], and refer to the original work for the general version. To this end, we recall that the continuous nonlinear n -width $d_{\mathcal{N}}(\mathcal{K}; \|\cdot\|_{\mathcal{X}})$ [39] of a subset $\mathcal{K} \subset \mathcal{X}$, with $(\mathcal{X}, \|\cdot\|_{\mathcal{X}})$ Banach, is defined as the optimal reconstruction error,

$$d_{\mathcal{N}}(\mathcal{K}; \|\cdot\|_{\mathcal{X}}) = \inf_{(a, M)} \sup_{f \in \mathcal{K}} \|f - M(a(f))\|_{\mathcal{X}},$$

where the infimum is over all encoder/decoder pairs (a, M) , consisting of a continuous map $a : \mathcal{K} \rightarrow \mathbb{R}^n$ and general map $M : \mathbb{R}^n \rightarrow \mathcal{X}$.

To derive lower n -width bounds, we consider spaces of nonlinear Lipschitz functionals $\Psi^\dagger \in \mathfrak{F}_d$, where d denotes the spatial dimension of the input functions. More, precisely define

$$\mathfrak{F}_d = \{\Psi^\dagger : L^2([-1, 1]^d) \rightarrow \mathbb{R} \mid \|\Psi^\dagger(u)\|_{\text{Lip}} \leq 1\},$$

with

$$\|\Psi^\dagger\|_{\text{Lip}} := \sup_{u \in L^2} |\Psi^\dagger(u)| + \sup_{u, v \in L^2} \frac{|\Psi^\dagger(u) - \Psi^\dagger(v)|}{\|u - v\|_{L^2}}.$$

Given a smoothness parameter $s > 0$, we consider approximation of $\Psi^\dagger \in \mathfrak{F}_d$, uniformly over a compact set of input functions $K^s \subset L^2([-1, 1]^d)$, obtained as follows: we expand input functions $f \in L^2([-1, 1]^d)$ in a Legendre expansion, $f(x) = \sum_{k \in \mathbb{N}^d} \hat{f}_k P_k(x)$, and consider functionals defined on the ‘‘Sobolev’’ ball,

$$K^s := \left\{ f \in L^2([-1, 1]^d) \left| \sum_{k \in \mathbb{N}^d} |k|^{2s} \hat{f}_k^2 \leq 1 \right. \right\}.$$

We measure the approximation error between $\Psi^\dagger, \Psi : K^s \subset L^2 \rightarrow \mathbb{R}$ with respect to the supremum norm over K^s ,

$$\|\Psi^\dagger - \Psi\|_{C(K^s)} = \sup_{u \in K^s} |\Psi^\dagger(u) - \Psi(u)|.$$

It follows from the main results of [97] that the continuous nonlinear n -widths of the set of functionals $\mathfrak{F}_d$ decay only (poly-)logarithmically, as

$$d_n(\mathfrak{F}_d)_{C(K^s)} \sim \log(n)^{-s/d}.$$

In particular, to achieve uniform approximation accuracy $\varepsilon > 0$ with a *continuous* encoder/decoder pair (a, M) , requires at least

$$(5.9) \quad n \gtrsim \exp\left(c\varepsilon^{-d/s}\right),$$

parameters. This last lower bound should be compared with (5.8) (for $k = 1$); the lower n -width bound (5.9) would imply (5.8) under the assumption that the architecture of $\psi = \psi(\cdot; \theta)$ was fixed and assuming that the weight assignment $\Psi^\dagger \mapsto \theta_{\Psi^\dagger}$ from the functional $\Psi^\dagger$ and the optimal tuning of neural network parameters $\theta_{\Psi^\dagger}$ was continuous. The latter assumption may not be satisfied if parameters are optimized using gradient descent.

Curse of Parametric Complexity. Given the result outlined in the previous paragraph, one may wonder if the pessimistic bound (5.9) and (5.7), i.e. the ‘‘curse of parametric complexity’’, can be broken by (a) a dis-continuous weight assignment, and (b) an adaptive choice of architecture optimized for specific $\Psi^\dagger$. This question has been raised in [72, 78].

It turns out that, with operator learning frameworks such as DeepONet, FNO or PCA-Net, and relying on standard neural network architectures, it is not possible to overcome the curse of parametric complexity when considering approximation on the full class of Lipschitz continuous or Fréchet differentiable operators. We mention

the following illustrative result for DeepONet, which follows from [78, Example 2.17]:

Proposition 5.4 (Curse of Parametric Complexity). Let $\mathfrak{D} \subset \mathbb{R}^d$ be a domain. Let $k \in \mathbb{N}$ be given, and consider the compact set of input functions,

$$K = \{u \in C^k(\mathfrak{D}) \mid \|u\|_{C^k} \leq 1\} \subset \mathcal{U} := C(\mathfrak{D}).$$

Fix $\alpha > 2 + \frac{k}{d}$. Then for any $r \in \mathbb{N}$, there exists a r -times Fréchet differentiable functional $\Psi^\dagger : \mathcal{U} \rightarrow \mathbb{R}$ and constant $c, \bar{\varepsilon} > 0$, such that approximation to accuracy $\varepsilon \leq \bar{\varepsilon}$ by a DeepONet $\Psi : \mathcal{U} \rightarrow \mathbb{R}$ with ReLU activation,

$$(5.10) \quad \sup_{u \in K} |\Psi^\dagger(u) - \Psi(u)| \leq \varepsilon,$$

with linear encoder $\mathcal{E}$ and neural network ψ , implies complexity bound

$$(5.11) \quad \text{size}(\psi) \geq \exp(c\varepsilon^{-1/\alpha r}).$$

Here $c, \bar{\varepsilon} > 0$ are constants depending only on k, α and r . ◇

As mentioned above, analogous lower complexity bounds can be obtained for PCA-Net, Fourier neural operator and many other architectures, and the more general version of this lower complexity bound applies to Sobolev input functions and beyond. We refer to [78] for a detailed discussion.

Remark 5.5. In [78], no attempt was made to optimize the exponent of ε^{-1} in (5.11). It would be interesting to know whether the appearance of the degree of operator Fréchet differentiability, i.e. the parameter r , in the lower bound is merely an artifact of the proof in [78]. The back-of-the-envelope calculation leading to (5.7) indicates that r should not appear in the exponent, and that the factor $\alpha = k/d + 2 + \delta$ could be replaced by $k/d + \delta$. ◇

Breaking the Curse of Parametric Complexity with Non-Standard Architectures. As mentioned in a previous section, there exist non-standard neural network architectures which either employ non-standard activations [109, 129, 143], or impose a non-standard “three-dimensional” connectivity [148] which can overcome the curse of dimensionality in finite dimensions. In particular, encoder-decoder-nets based on such non-standard architectures can achieve neural network approximation error $\mathcal{E}_\psi \leq \varepsilon$ with a complexity (as measured by the number of tunable degrees of freedom) that grows much slower than the rough scaling we considered in (5.6). Based on such architectures, it has recently been shown [123] that DeepONets can achieve approximation rates for general Lipschitz and Hölder continuous operators which break the curse of parametric complexity implied by (5.11). In fact, such architectures achieve algebraic expression rate bounds for general Lipschitz and Hölder continuous operators.

5.3.4. Sample Complexity Results. There is a rapidly growing body of work on the approximation theory of operator learning with focus on parametric complexity. The complementary question of the sample complexity of operator learning, i.e. how many samples are needed to achieve a given approximation accuracy, has not received as much attention. The work described in subsection 5.1 addresses this question in the setting of learning linear operators, and the question is also addressed in subsection 5.2 for holomorphic operators. We now develop this subject further. Of particular note in the general Lipschitz setting of this subsection is the paper [90], as well as related recent work in [23], which studies the nonparametric error estimation of Lipschitz operators for general encoder-decoder-net architectures. In [90], non-asymptotic upper bounds for the generalization error of empirical risk minimizers on suitable classes of operator networks are derived. The results are

stated in a Bochner $L^2(\mu)$ setting with input functions drawn from a probability measure μ , and variants are derived for general (Lipschitz) encoder/decoder pairs, for fixed basis encoder/decoder pairs, and for PCA encoder/decoder pairs. The analysis underlying the approximation error estimates in [90] is based on a combination of best-approximation error estimates (parametric complexity) which are combined with statistical learning theory to derive sample complexity bounds.

For detailed results applicable to more general settings, we refer the reader to [90]. Here, we restrict attention to a representative result for fixed basis encoder/decoder pair [90, Corollary 3], obtained by projection onto a trigonometric basis.

To state this simplified result, consider a Lipschitz operator $\Psi^\dagger : \mathcal{U} \rightarrow \mathcal{V}$, mapping between spaces $\mathcal{U}, \mathcal{V} = L^2([-1, 1]^d)$. We assume that there exists a constant $C > 0$, such that the probability measure $\mu \in \mathcal{P}(\mathcal{U})$ and its push-forward $\Psi^\dagger_{\#}\mu \in \mathcal{P}(\mathcal{V})$ are supported on periodic, continuously differentiable functions belonging to the set,

$$K := \{u \in L^2([-1, 1]^d) \mid u \text{ is periodic, } \|u\|_{C^{k,\alpha}} \leq C\}.$$

Then the squared approximation error

$$\mathcal{E}^2 := \mathbb{E}_{\text{data}} \mathbb{E}_{u \sim \mu} \|D_{\mathcal{Y}} \circ \psi \circ E_{\mathcal{X}}(u) - \Psi^\dagger(u)\|_{L^2}^2$$

satisfies the upper bound,

$$(5.12) \quad \mathcal{E}^2 \lesssim d_{\mathcal{V}}^{\frac{4+d_{\mathcal{U}}}{2+d_{\mathcal{U}}}} N^{-\frac{2}{2+d_{\mathcal{U}}}} \log^6(n) + d_{\mathcal{U}}^{-\frac{2s}{d}} + d_{\mathcal{V}}^{-\frac{2s}{d}},$$

where N is the number of samples and $s = k + \alpha$ is the smoothness on the input and output spaces. The neural network ψ is a ReLU network of depth L and width p , satisfying (up to logarithmic terms),

$$Lp \sim d_{\mathcal{V}}^{\frac{d_{\mathcal{U}}}{4+2d_{\mathcal{U}}}} N^{\frac{d_{\mathcal{U}}}{4+2d_{\mathcal{U}}}}.$$

Comparing with (5.4) and (5.5), we can identify the last two terms in (5.12) as the encoding and reconstruction errors. The first term corresponds to a combination of neural network approximation and generalization errors.

To ensure that the total error $\mathcal{E} \leq \varepsilon$ is below accuracy threshold ε , we first choose $d_{\mathcal{U}}, d_{\mathcal{V}} \sim \varepsilon^{-d/s}$, consistent with our discussion in subsection 5.3.3. And according to the above estimate, we choose a number of samples of roughly the size $N \sim \varepsilon^{-(2+d_{\mathcal{U}})/2}$. Note that, once more, the additional ε -dependency of $d_{\mathcal{U}}$ implies that

$$N \gtrsim \exp\left(c\varepsilon^{-d/s}\right),$$

exhibits an exponential curse of complexity. This time, the curse is reflected by an exponential number of samples N that are required to achieve accuracy ε , rather than the parametric complexity. In turn, this implies that the size of the product Lp of the depth L and width p of the neural network ψ satisfies the lower bound,

$$Lp \gtrsim \exp\left(c\varepsilon^{-d/s}\right),$$

consistent with the expected curse of parametric complexity, (5.7). It is likely that the results of [90] cannot be substantially improved in the considered setting of Lipschitz operators. Extending their work to a slightly different setting, the authors of [90] also raise the question of low dimensional structure in operator learning, and derive error bounds decaying with a fast rate under suitable conditions, relying on low-dimensional latent structure of the input space.

In a related direction, the authors of [62] provide estimates on the Rademacher complexity of FNO. Generalization error estimates are derived based on these Rademacher complexity estimates, and the theoretical insights are compared with

the empirical generalization error and the proposed capacity of FNO, in numerical experiments. Out-of-distribution risk bounds for neural operators with focus on the Helmholtz equation are discussed in depth in [13].

We finally mention the recent work [100], where a connection is made between the number of available samples n and the required size of the DeepONet reconstruction dimension d_V . It is shown that when only noisy measurements are available, a scaling of the number of trunk basis functions $d_V \gtrsim \sqrt{n}$ is required to achieve accurate approximation.

5.4. Structure Beyond Smoothness. The results summarized in the previous sections indicate that, when relying on standard neural network architectures, *efficient* operator learning on general spaces of Lipschitz continuous, or Fréchet differentiable, operators may not be possible: the class of all such operators on infinite-dimensional Banach spaces is arguably too rich, and operator learning on this class suffers from a curse of parametric complexity, requiring exponential model sizes of the form $\gtrsim \exp(c\varepsilon^{-\gamma})$.

This is in contrast to operator learning for $(\mathbf{b}, \varepsilon)$ -holomorphic operators, for which approximation to accuracy ε is possible with a parametric complexity $O(\varepsilon^{-\gamma})$ scaling only algebraically in ε^{-1} . In this case, the curse of parametric complexity is broken by the extraordinary amount of smoothness of the underlying operators, going far beyond Lipschitz continuity or Fréchet differentiability.

These contrasting results rely only on the smoothness of the approximated operator: Is such smoothness the deciding factor for the practical success of operator learning methodologies? While we currently cannot provide a theoretical answer to this important question, we finally would like to mention several approximation theoretic results addressing how operator learning frameworks can break the curse of parametric complexity by leveraging structure beyond holomorphy.

Operator Barron Spaces. A celebrated result in the study of shallow neural networks on finite-dimensional spaces is Barron’s discovery [10] of a function space on which *dimension-independent* Monte-Carlo approximation rates $O(1/\sqrt{n})$ can be obtained. In particular, the approximation, by shallow neural networks, of functions belonging to this Barron class does not suffer from the well-known curse of dimensionality.

In the recent paper [64], a suitable generalization of the Barron spaces is introduced, and it is shown that Monte-Carlo approximation rates $O(1/\sqrt{n})$ can be obtained even in this infinite-dimensional setting, under precisely specified conditions. Quantitative error estimates (convergence rates) for the approximation of nonlinear operators are obtained by extending earlier results [8, 41, 42] from the finite-dimensional setting $f : \mathbb{R}^d \rightarrow \mathbb{R}$ to the vector-valued and infinite-dimensional case $f : \mathcal{U} \rightarrow \mathcal{V}$, where $\mathcal{U}$ and $\mathcal{V}$ are Banach spaces.

The operator Barron spaces identified in [64] represent a general class of operators, distinct from the holomorphic operators discussed in a previous section, which allow for efficient approximation by a class of “shallow neural operators”. Unfortunately, a priori, it is unclear which operators of practical interest belong to this class, leaving the connection between these theoretical results and the practically observed efficiency of neural operator somewhat tenuous. In passing, we also mention the operator reproducing kernel Hilbert spaces (RKHS) considered in the context of the random feature model in [101], for which similar Monte-Carlo convergence rates have been derived in [76].

Representation Formulae and Emulation of Numerical Methods. To conclude our discussion of complexity and error bounds, we mention work focused on additional structure, separate from smoothness and the above-mentioned idea of

Barron spaces, which can be leveraged by operator learning frameworks to achieve efficient approximation: these include operators with explicit representation formulae, and operators for which efficient approximation by traditional numerical schemes is possible. Such representations by classical methods can often be efficiently emulated by operator learning methodologies, resulting in error and complexity estimates that beat the curse of parametric complexity.

The complexity estimates for DeepONets in [37] are mostly based on explicit representation of the solution; most prominently, this is achieved via the Cole-Hopf transformation for the viscous Burgers equation.

Results employing emulation of numerical methods to prove that operator learning frameworks such as DeepONet, FNO and PCA-Net can overcome the curse of parametric complexity for specific operators of interest can be found in [67, 72, 74, 94]; specifically, such results have e.g. been obtained for the Darcy flow equation, the Navier-Stokes equations, reaction-diffusion equations and the inviscid Burgers equation. For the solution operators associated with these PDEs, it has been shown that operator learning frameworks can achieve approximation accuracy ε with a total number of tunable degrees of freedom which either only scales algebraically in ε , i.e. with $\text{size}(\psi) = O(\varepsilon^{-\gamma})$, or scales only logarithmically, $\text{size}(\psi) = O(|\log \varepsilon|^\gamma)$ in certain settings [94]. This should be contrasted with the general curse of dimensionality (5.7). It is expected that the underlying ideas apply to many other PDEs.

Results in this direction are currently only available for very specific operators, and an abstract characterization of the relevant structure that can be exploited by operator learning frameworks is not available. First steps towards a more general theory have been proposed in [120], where generic bounds for operator learning are derived, relating the approximation error for physics-informed neural networks (PINNs) and operator learning architectures such as DeepONets and FNOs.

5.4.1. Discussion. Ultimately, the overarching theme behind many of the above cited results is that neural operators, or neural networks more generally, can efficiently emulate numerical algorithms, which either result from bespoke numerical methods or are a consequence of explicit representation formulae. The total complexity of a neural network emulator, and reflected by its size, is composed of the complexity of the emulated numerical algorithm and an additional overhead cost of emulating this algorithm by a neural network (translation to neural network weights). From an approximation-theoretic point of view, it could be conjectured that, for a suitable definition of “numerical algorithm”, neural networks can efficiently approximate *all numerical algorithms*, hence implying efficient approximation of a great variety of operators, excluding only those operators for which no efficient numerical algorithms exist. Formalizing a suitable notion of numerical algorithm and proving that neural networks can efficiently emulate any such algorithm would be of interest and could provide a general way for proving algebraic expression rate bound for a general class of operators that can be approximated by a numerical method with algebraic memory and run-time complexity (i.e. any “reasonable” approximation method).

6. CONCLUSIONS

Neural operator architectures employ neural networks to approximate nonlinear operators mapping between Banach spaces of functions. Such operators often arise from physical models which are expressed as partial differential equations (PDEs). Despite their empirical success in a variety of applications, our theoretical understanding of neural operators remains incomplete. This review article summarizes

recent progress and the current state of our theoretical understanding of neural operators, focusing on an approximation theoretic point of view.

The starting point of the theoretical analysis is universal approximation. Very general universal approximation results are now available for many of the proposed neural operator architectures. These results demonstrate that, given a sufficient number of parameters, neural operators can approximate a very wide variety of infinite-dimensional operators, providing a theoretical underpinning for diverse applications. Such universal approximation is arguably a necessary but not sufficient condition for the success of these architectures. In particular, universal approximation is inherently qualitative and does not guarantee that approximation to a desired accuracy is feasible at a practically realistic model size.

A number of more recent works thus aim to provide quantitative bounds on the required model size and the required number of input-/output-samples to achieve a desired accuracy ϵ . Most such results consider one of three settings: general Lipschitz (or Fréchet differentiable) operators, holomorphic operators, or specific PDE operators. While Lipschitz operators are a natural and general class to consider, it turns out that approximation to error ϵ with standard architectures requires an exponential (in ϵ^{-1}) number of tunable parameters, bringing into question whether operator learning at this level of generality is possible. In contrast, the class of holomorphic operators allows for complexity bounds that scale only algebraically in ϵ^{-1} , both in terms of models size as well as sample complexity. Holomorphic operators represent a general class of operators of practical interest, for which rigorous approximation theory has been developed building on convergent (generalized) polynomial expansions.

Going beyond notions of operator smoothness, it has been shown that operator learning frameworks can leverage intrinsic structure of (PDE-) operators to achieve algebraic convergence rates in theory; this intrinsic structure is distinct from holomorphy. Available results in this direction currently rely a case-by-case analysis and often leverage emulation of traditional numerical methods. The authors of the present article view the development of a general approximation theory, including a characterization of the relevant structure that can be leveraged by neural operators, as one of the great challenges of this field.

Acknowledgments. The authors are grateful to Dima Burov and Edo Calvello for creating Figures 1a and 1b, 2a, 2b respectively. They are also grateful to Nick Nelsen for reading, and commenting on, a draft of the paper. NBK is grateful to the NVIDIA Corporation for full time employment. SL is grateful for support from the Swiss National Science Foundation, Postdoc.Mobility grant P500PT-206737. AMS is grateful for support from a Department of Defense Vannevar Bush Faculty Fellowship.

REFERENCES

- [1] J. Abernethy, F. Bach, T. Evgeniou, and J.-P. Vert. A new approach to collaborative filtering: Operator estimation with spectral regularization. *The Journal of Machine Learning Research*, 10:803–826, 2009.
- [2] E. M. Achour, A. Foucault, S. Gerchinovitz, and F. Malgouyres. A general approximation lower bound in L^p norm, with applications to feed-forward neural networks. In *Advances in Neural Information Processing Systems*, 2022.
- [3] B. Adcock, S. Brugiapaglia, N. Dexter, and S. Moraga. Near-optimal learning of Banach-valued, high-dimensional functions via deep neural networks. *arXiv preprint arXiv:2211.12633*, 2022.
- [4] B. Adcock, S. Brugiapaglia, N. Dexter, and S. Moraga. On efficient algorithms for computing near-best polynomial approximations to high-dimensional, Hilbert-valued functions from limited samples. *arXiv preprint arXiv:2203.13908*, 2022.

- [5] B. Adcock, S. Brugiapaglia, and C. G. Webster. *Sparse polynomial approximation of high-dimensional functions*, volume 25. SIAM, 2022.
- [6] B. Adcock, N. Dexter, and S. Moraga. Optimal approximation of infinite-dimensional holomorphic functions. *arXiv preprint arXiv:2305.18642*, 2023.
- [7] B. Adcock, N. Dexter, and S. Moraga. Optimal approximation of infinite-dimensional holomorphic functions ii: recovery from iid pointwise samples. *arXiv preprint arXiv:2310.16940*, 2023.
- [8] F. Bach. Breaking the curse of dimensionality with convex neural networks. *The Journal of Machine Learning Research*, 18(1):629–681, 2017.
- [9] M. Bachmayr and A. Cohen. Kolmogorov widths and low-rank approximations of parametric elliptic PDEs. *Mathematics of Computation*, 86(304):701–724, 2017.
- [10] A. R. Barron. Universal approximation bounds for superpositions of a sigmoidal function. *IEEE Transactions on Information theory*, 39(3):930–945, 1993.
- [11] F. Bartolucci, E. de Bézenac, B. Raonić, R. Molinaro, S. Mishra, and R. Alaifari. Are neural operators really neural operators? Frame theory meets operator learning. *arXiv preprint arXiv:2305.19913*, 2023.
- [12] P. Batlle, M. Darcy, B. Hosseini, and H. Owhadi. Kernel methods are competitive for operator learning. *Journal of Computational Physics*, 496:112549, 2024.
- [13] J. A. L. Benitez, T. Furuya, F. Faucher, A. Kratsios, X. Tricoche, and M. V. de Hoop. Out-of-distributional risk bounds for neural operators with applications to the Helmholtz equation. *arXiv preprint arXiv:2301.11509*, 2023.
- [14] K. Bhattacharya, B. Hosseini, N. B. Kovachki, and A. M. Stuart. Model reduction and neural networks for parametric PDEs. *The SMAI Journal of Computational Mathematics*, 7:121–157, 2021.
- [15] H. Bolcskei, P. Grohs, G. Kutyniok, and P. Petersen. Optimal approximation with sparsely connected deep neural networks. *SIAM Journal on Mathematics of Data Science*, 1(1):8–45, 2019.
- [16] B. Bonev, T. Kurth, C. Hundt, J. Pathak, M. Baust, K. Kashinath, and A. Anandkumar. Spherical fourier neural operators: learning stable dynamics on the sphere. In *Proceedings of the 40th International Conference on Machine Learning*, 2023.
- [17] N. Boullé, C. J. Earls, and A. Townsend. Data-driven discovery of Green’s functions with human-understandable deep learning. *Scientific reports*, 12(1):4824, 2022.
- [18] N. Boullé, S. Kim, T. Shi, and A. Townsend. Learning Green’s functions associated with time-dependent partial differential equations. *The Journal of Machine Learning Research*, 23(1):9797–9830, 2022.
- [19] N. Boullé and A. Townsend. Learning elliptic partial differential equations with randomized linear algebra. *Foundations of Computational Mathematics*, 23(2):709–739, 2023.
- [20] S. L. Brunton and J. N. Kutz. Machine learning for partial differential equations. *arXiv preprint arXiv:2303.17078*, 2023.
- [21] J. Castro. The Kolmogorov infinite dimensional equation in a Hilbert space via deep learning methods. *Journal of Mathematical Analysis and Applications*, 527(2):127413, 2023.
- [22] J. Castro, C. Muñoz, and N. Valenzuela. The Calderón’s problem via DeepONets. *arXiv preprint arXiv:2212.08941*, 2022.
- [23] K. Chen, C. Wang, and H. Yang. Deep operator learning lessens the curse of dimensionality for pdes. *arXiv preprint arXiv:2301.12227*, 2023.
- [24] T. Chen and H. Chen. Universal approximation to nonlinear operators by neural networks with arbitrary activation functions and its application to dynamical systems. *IEEE Transactions on Neural Networks*, 6(4):911–917, 1995.
- [25] A. Chkifa, A. Cohen, R. DeVore, and C. Schwab. Sparse adaptive taylor approximation algorithms for parametric and stochastic elliptic PDEs. *ESAIM: Mathematical Modelling and Numerical Analysis*, 47(1):253–280, 2013.
- [26] A. Chkifa, A. Cohen, and C. Schwab. Breaking the curse of dimensionality in sparse polynomial approximation of parametric PDEs. *Journal de Mathématiques Pures et Appliquées*, 103(2):400–428, 2015.
- [27] A. Cohen, W. Dahmen, O. Mula, and J. Nichols. Nonlinear reduced models for state and parameter estimation. *SIAM/ASA Journal on Uncertainty Quantification*, 10(1):227–267, 2022.
- [28] A. Cohen and R. DeVore. Approximation of high-dimensional parametric PDEs. *Acta Numerica*, 24:1–159, 2015.
- [29] A. Cohen, R. DeVore, and C. Schwab. Convergence rates of best n-term galerkin approximations for a class of elliptic SPDEs. *Foundations of Computational Mathematics*, 10(6):615–646, 2010.

- [30] A. Cohen, R. Devore, and C. Schwab. Analytic regularity and polynomial approximation of parametric and stochastic elliptic PDEs. *Analysis and Applications*, 9(01):11–47, 2011.
- [31] M. J. Colbrook and A. Townsend. Rigorous data-driven computation of spectral properties of Koopman operators for dynamical systems. *Communications on Pure and Applied Mathematics*, 77(1):221–283, 2024.
- [32] S. Cotter, G. Roberts, A. Stuart, and D. White. MCMC methods for functions: Modifying old algorithms to make them faster. *Statistical Science*, 28, 02 2012.
- [33] C. Crambes and A. Mas. Asymptotics of prediction in functional linear regression with functional outputs. *Bernoulli*, 19(5B):2627 – 2651, 2013.
- [34] G. Cybenko. Approximation by superpositions of a sigmoidal function. *Mathematics of control, signals and systems*, 2(4):303–314, 1989.
- [35] M. De Hoop, D. Z. Huang, E. Qian, and A. M. Stuart. The cost-accuracy trade-off in operator learning with neural networks. *Journal of Machine Learning*, 1:3:299–341, 2022.
- [36] M. V. de Hoop, N. B. Kovachki, N. H. Nelsen, and A. M. Stuart. Convergence rates for learning linear operators from noisy data. *SIAM/ASA Journal on Uncertainty Quantification*, 11(2):480–513, 2023.
- [37] B. Deng, Y. Shin, L. Lu, Z. Zhang, and G. E. Karniadakis. Convergence rate of DeepONets for learning operators arising from advection-diffusion equations. *arXiv preprint arXiv:2102.10621*, 2021.
- [38] R. A. DeVore. Nonlinear approximation. *Acta Numerica*, 7:51–150, 1998.
- [39] R. A. DeVore, R. Howard, and C. Micchelli. Optimal nonlinear approximation. *Manuscripta mathematica*, 63:469–478, 1989.
- [40] R. A. DeVore and G. G. Lorentz. *Constructive approximation*, volume 303. Springer Science & Business Media, 1993.
- [41] W. E, C. Ma, and L. Wu. The Barron space and the flow-induced function spaces for neural network models. *Constructive Approximation*, 55(1):369–406, 2022.
- [42] W. E and S. Wojtowytsch. Representation formulas and pointwise properties for Barron functions. *Calculus of Variations and Partial Differential Equations*, 61(2):1–37, 2022.
- [43] P. Enflo. A counterexample to the approximation problem in Banach spaces. *Acta Mathematica*, 130:309–316, 1973.
- [44] L. C. Evans. *Partial differential equations*, volume 19. American Mathematical Soc., 2010.
- [45] N. R. Franco, S. Fresca, A. Manzoni, and P. Zunino. Approximation bounds for convolutional neural networks in operator learning. *Neural Networks*, 161:129–141, 2023.
- [46] T. Furuya, M. Puthawala, M. Lassas, and M. V. de Hoop. Globally injective and bijective neural operators. In *Thirty-eight Conference on Neural Information Processing Systems*, 2024.
- [47] L. Galimberti, A. Kratsios, and G. Livieri. Designing universal causal deep learning models: The case of infinite-dimensional dynamical systems from stochastic analysis. *arXiv preprint arXiv:2210.13300*, 2022.
- [48] C. R. Gin, D. E. Shea, S. L. Brunton, and J. N. Kutz. Deepgreen: Deep learning of Green’s functions for nonlinear boundary value problems. *arXiv preprint arXiv:2101.07206*, 2020.
- [49] I. Goodfellow, Y. Bengio, and A. Courville. *Deep Learning*. MIT press, 2016.
- [50] J. K. Gupta and J. Brandstetter. Towards multi-spatiotemporal-scale generalized PDE modeling. *arXiv preprint arXiv:2209.15616*, 2022.
- [51] M. Hairer, A. M. Stuart, and S. J. Vollmer. Spectral gaps for a Metropolis–Hastings algorithm in infinite dimensions. *The Annals of Applied Probability*, 24(6):2455–2490, 2014.
- [52] L. Herrmann, C. Schwab, and J. Zech. Neural and GPC operator surrogates: Construction and expression rate bounds. *arXiv preprint arXiv:2207.04950*, 2022.
- [53] J. S. Hesthaven and S. Ubbiali. Non-intrusive reduced order modeling of nonlinear problems using neural networks. *Journal of Computational Physics*, 363:55–78, 2018.
- [54] M. Hinze, R. Pinnau, M. Ulbrich, and S. Ulbrich. *Optimization with PDE constraints*, volume 23. Springer Science & Business Media, 2008.
- [55] K. Hornik, M. Stinchcombe, and H. White. Multilayer feedforward networks are universal approximators. *Neural networks*, 2(5):359–366, 1989.
- [56] N. Hua and W. Lu. Basis operator network: A neural network-based model for learning nonlinear operators via neural basis. *Neural Networks*, 164:21–37, 2023.
- [57] D. Z. Huang, N. H. Nelsen, and M. Trautner. An operator learning perspective on parameter-to-observable maps. *arXiv preprint arXiv:2402.06031*, 2024.
- [58] D. Z. Huang, K. Xu, C. Farhat, and E. Darve. Learning constitutive relations from indirect observations using deep neural networks. *Journal of Computational Physics*, 416:109491, 2020.
- [59] J. Jin, Y. Lu, J. Blanchet, and L. Ying. Minimax optimal kernel operator learning via multilevel training. In *Eleventh International Conference on Learning Representations*, 2022.

- [60] P. Jin, S. Meng, and L. Lu. Mionet: Learning multiple-input operators via tensor product. *SIAM Journal on Scientific Computing*, 44(6):A3490–A3514, 2022.
- [61] J. Kaipio and E. Somersalo. *Statistical and Computational Inverse Problems*, volume 160. Springer Science & Business Media, 2006.
- [62] T. Kim and M. Kang. Bounding the rademacher complexity of Fourier neural operator. *arXiv preprint arXiv:2209.05150*, 2022.
- [63] I. Klebanov and T. Sullivan. Aperiodic table of modes and maximum a posteriori estimators. *arXiv preprint arXiv:2306.16278*, 2023.
- [64] Y. Korolev. Two-layer neural networks with values in a banach space. *SIAM Journal on Mathematical Analysis*, 54(6):6358–6389, 2022.
- [65] V. Kostic, P. Novelli, A. Maurer, C. Ciliberto, L. Rosasco, and M. Pontil. Learning dynamical systems via Koopman operator regression in reproducing kernel hilbert spaces. In *Thirty-sixth Annual Conference on Neural Information Processing Systems*, 2022.
- [66] N. B. Kovachki. *Machine Learning and Scientific Computing*. PhD thesis, California Institute of Technology, 2022.
- [67] N. B. Kovachki, S. Lanthaler, and S. Mishra. On universal approximation and error bounds for Fourier neural operators. *Journal of Machine Learning Research*, 22(1), 2021.
- [68] N. B. Kovachki, Z. Li, B. Liu, K. Azizzadenesheli, K. Bhattacharya, A. Stuart, and A. Anandkumar. Neural operator: Learning maps between function spaces with applications to PDEs. *Journal of Machine Learning Research*, 24(89), 2023.
- [69] B. Kramer, B. Peherstorfer, and K. E. Willcox. Learning nonlinear reduced models from data with operator inference. *Annual Review of Fluid Mechanics*, 56(1):521–548, 2024.
- [70] G. Kutyniok, P. Petersen, M. Raslan, and R. Schneider. A theoretical analysis of deep neural networks and parametric PDEs. *Constructive Approximation*, 55(1):73–125, 2022.
- [71] R. Lam, A. Sanchez-Gonzalez, M. Willson, P. Wirsberger, M. Fortunato, F. Alet, S. Ravuri, T. Ewalds, Z. Eaton-Rosen, W. Hu, A. Merose, S. Hoyer, G. Holland, O. Vinyals, J. Stott, A. Pritzel, S. Mohamed, and P. Battaglia. Learning skillful medium-range global weather forecasting. *Science*, 382(6677):1416–1421, 2023.
- [72] S. Lanthaler. Operator learning with PCA-Net: Upper and lower complexity bounds. *Journal of Machine Learning Research*, 24(318), 2023.
- [73] S. Lanthaler, Z. Li, and A. M. Stuart. The nonlocal neural operator: Universal approximation. *arXiv preprint arXiv:2304.13221*, 2023.
- [74] S. Lanthaler, S. Mishra, and G. E. Karniadakis. Error estimates for DeepONets: A deep learning framework in infinite dimensions. *Transactions of Mathematics and Its Applications*, 6(1), 2022.
- [75] S. Lanthaler, R. Molinaro, P. Hadorn, and S. Mishra. Nonlinear reconstruction for operator learning of PDEs with discontinuities. In *Eleventh International Conference on Learning Representations*, 2023.
- [76] S. Lanthaler and N. H. Nelsen. Error bounds for learning with vector-valued random features. In *Thirty-seventh Conference on Neural Information Processing Systems*, 2023.
- [77] S. Lanthaler, T. K. Rusch, and S. Mishra. Neural oscillators are universal. In *Thirty-seventh Conference on Neural Information Processing Systems*, 2023.
- [78] S. Lanthaler and A. M. Stuart. The curse of dimensionality in operator learning. *arXiv preprint arXiv:2306.15924*, 2023.
- [79] K. Lee and K. T. Carlberg. Model reduction of dynamical systems on nonlinear manifolds using deep convolutional autoencoders. *Journal of Computational Physics*, 404:108973, 2020.
- [80] Z. Lei, L. Shi, and C. Zeng. Solving parametric partial differential equations with deep rectified quadratic unit neural networks. *Journal of Scientific Computing*, 93(3):80, 2022.
- [81] Q. Li, F. Dietrich, E. M. Boltt, and I. G. Kevrekidis. Extended dynamic mode decomposition with dictionary learning: A data-driven adaptive spectral decomposition of the Koopman operator. *Chaos: An Interdisciplinary Journal of Nonlinear Science*, 27(10):103111, 2017.
- [82] Z. Li, N. B. Kovachki, K. Azizzadenesheli, B. Liu, K. Bhattacharya, A. M. Stuart, and A. Anandkumar. Neural operator: Graph kernel network for partial differential equations. *arXiv preprint arXiv:2003.03485*, 2020.
- [83] Z. Li, N. B. Kovachki, K. Azizzadenesheli, B. Liu, K. Bhattacharya, A. M. Stuart, and A. Anandkumar. Fourier neural operator for parametric partial differential equations. In *Ninth International Conference on Learning Representations*, 2021.
- [84] Z. Li, N. B. Kovachki, K. Azizzadenesheli, B. Liu, A. M. Stuart, K. Bhattacharya, and A. Anandkumar. Multipole graph neural operator for parametric partial differential equations. In *Thirty-fourth Annual Conference on Neural Information Processing Systems*, 2020.
- [85] S. Liang, L. Lyu, C. Wang, and H. Yang. Reproducing activation function for deep learning. *arXiv preprint arXiv:2101.04844*, 2021.

- [86] J. Lindenstrauss and L. Tzafriri. *Classical Banach Spaces I: Sequence Spaces*. Ergebnisse der Mathematik und ihrer Grenzgebiete. 2. Folge. Springer Berlin Heidelberg, 2013.
- [87] J. Ling, A. Kurzwaski, and J. Templeton. Reynolds averaged turbulence modelling using deep neural networks with embedded invariance. *Journal of Fluid Mechanics*, 807:155–166, 2016.
- [88] P. Lippe, B. Veeling, P. Perdikaris, R. Turner, and J. Brandstetter. PDE-refiner: Achieving accurate long rollouts with neural PDE solvers. In *Thirty-seventh Annual Conference on Neural Information Processing Systems*, 2024.
- [89] O. Litany, T. Remez, E. Rodola, A. Bronstein, and M. Bronstein. Deep functional maps: Structured prediction for dense shape correspondence. In *Proceedings of the IEEE international conference on computer vision*, pages 5659–5667, 2017.
- [90] H. Liu, H. Yang, M. Chen, T. Zhao, and W. Liao. Deep nonparametric estimation of operators between infinite dimensional spaces. *Journal of Machine Learning Research*, 25(24):1–67, 2024.
- [91] X. Liu, S. Tian, F. Tao, and W. Yu. A review of artificial neural networks in the constitutive modeling of composite materials. *Composites Part B: Engineering*, 224:109152, 2021.
- [92] L. Lu, P. Jin, G. Pang, Z. Zhang, and G. E. Karniadakis. Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators. *Nature Machine Intelligence*, 3(3):218–229, 2021.
- [93] D. Luo, T. O’Leary-Roseberry, P. Chen, and O. Ghattas. Efficient PDE-constrained optimization under high-dimensional uncertainty using derivative-informed neural operators. *arXiv preprint arXiv:2305.20053*, 2023.
- [94] C. Marcati and C. Schwab. Exponential convergence of deep operator networks for elliptic partial differential equations. *SIAM Journal on Numerical Analysis*, 61(3):1513–1545, 2023.
- [95] H. Mhaskar. Local approximation of operators. *Applied and Computational Harmonic Analysis*, 64:194–228, 2023.
- [96] H. Mhaskar and T. Poggio. Function approximation by deep networks. *Communications on Pure and Applied Analysis*, 19(8):4085–4095, 2020.
- [97] H. N. Mhaskar and N. Hahm. Neural networks for functional approximation and system identification. *Neural Computation*, 9(1):143–159, 1997.
- [98] M. Mollenhauer, N. Mücke, and T. Sullivan. Learning linear operators: Infinite-dimensional regression as a well-behaved non-compact inverse problem. *arXiv preprint arXiv:2211.08875*, 2022.
- [99] J. Morton, F. D. Witherden, A. Jameson, and M. J. Kochenderfer. Deep dynamical modeling and control of unsteady fluid flows. In *Proceedings of the 32nd International Conference on Neural Information Processing Systems*, 2018.
- [100] A. Mukherjee and A. Roy. Size lowerbounds for deep operator networks. *arXiv preprint arXiv:2308.06338*, 2023.
- [101] N. H. Nelsen and A. M. Stuart. The random feature model for input-output maps between Banach spaces. *SIAM Journal on Scientific Computing*, 43(5):A3212–A3243, 2021.
- [102] T. O’Leary-Roseberry, P. Chen, U. Villa, and O. Ghattas. Derivative-informed neural operator: An efficient framework for high-dimensional parametric derivative learning. *Journal of Computational Physics*, 496:112555, 2024.
- [103] J. A. Opschoor, C. Schwab, and J. Zech. Exponential ReLU DNN expression of holomorphic maps in high dimension. *Constructive Approximation*, 55(1):537–582, 2022.
- [104] M. Ovsjanikov, M. Ben-Chen, J. Solomon, A. Butscher, and L. Guibas. Functional maps: A flexible representation of maps between shapes. *ACM Trans. Graph.*, 31(4), 2012.
- [105] D. Patel, D. Ray, M. R. Abdelmalik, T. J. Hughes, and A. A. Oberai. Variationally mimetic operator networks. *Computer Methods in Applied Mechanics and Engineering*, 419:116536, 2024.
- [106] R. G. Patel and O. Desjardins. Nonlinear integro-differential operator regression with neural networks. *arXiv preprint arXiv:1810.08552*, 2018.
- [107] R. G. Patel, N. A. Trask, M. A. Wood, and E. C. Cyr. A physics-informed operator regression framework for extracting data-driven continuum models. *Computer Methods in Applied Mechanics and Engineering*, 373:113500, 2021.
- [108] B. Peherstorfer and K. Willcox. Data-driven operator inference for nonintrusive projection-based model reduction. *Computer Methods in Applied Mechanics and Engineering*, 306:196–215, 2016.
- [109] A. Pinkus. Approximation theory of the mlp model in neural networks. *Acta Numerica*, 8:143–195, 1999.
- [110] M. Prasthofer, T. De Ryck, and S. Mishra. Variable-input deep operator networks. *arXiv preprint arXiv:2205.11404*, 2022.

- [111] E. Qian, I.-G. Farcas, and K. Willcox. Reduced operator inference for nonlinear partial differential equations. *SIAM Journal on Scientific Computing*, to appear, 2022.
- [112] E. Qian, B. Kramer, B. Peherstorfer, and K. Willcox. Lift & Learn: Physics-informed machine learning for large-scale nonlinear dynamical systems. *Physica D: Nonlinear Phenomena*, 406:132401, 2020.
- [113] M. Raghu and E. Schmidt. A survey of deep learning for scientific discovery. *arXiv preprint arXiv:2003.11755*, 2020.
- [114] A. Rahimi and B. Recht. Random features for large-scale kernel machines. In *Twenty-First Annual Conference on Neural Information Processing Systems*, 2007.
- [115] M. A. Rahman, Z. E. Ross, and K. Azzizadenesheli. U-NO: U-shaped neural operators. *Transactions on Machine Learning Research*, 2023.
- [116] J. O. Ramsay and C. Dalzell. Some tools for functional data analysis. *Journal of the Royal Statistical Society Series B: Statistical Methodology*, 53(3):539–561, 1991.
- [117] B. Raonic, R. Molinaro, T. Rohner, S. Mishra, and E. de Bezenac. Convolutional neural operators. In *ICLR 2023 Workshop on Physics for Machine Learning*, 2023.
- [118] B. Raonic, R. Molinaro, T. D. Ryck, T. Rohner, F. Bartolucci, R. Alaifari, S. Mishra, and E. de Bezenac. Convolutional neural operators for robust and accurate learning of PDEs. In *Thirty-seventh Conference on Neural Information Processing Systems*, 2023.
- [119] L. Rosasco, M. Belkin, and E. D. Vito. On learning with integral operators. *The Journal of Machine Learning Research*, 11:905–934, 2010.
- [120] T. D. Ryck and S. Mishra. Generic bounds on the approximation error for physics-informed (and) operator learning. In *Thirty-sixth Annual Conference on Neural Information Processing Systems*, 2022.
- [121] J. Sacks, W. J. Welch, T. J. Mitchell, and H. P. Wynn. Design and analysis of computer experiments. *Statistical Science*, 4(4):409 – 423, 1989.
- [122] J. Schmidt-Hieber. Nonparametric regression using deep neural networks with ReLU activation function. *The Annals of Statistics*, 48(4):1875 – 1897, 2020.
- [123] C. Schwab, A. Stein, and J. Zech. Deep operator network approximation rates for Lipschitz operators. *arXiv preprint arXiv:2307.09835*, 2023.
- [124] C. Schwab and J. Zech. Deep learning in high dimension: Neural network expression rates for generalized polynomial chaos expansions in UQ. *Analysis and Applications*, 17(01):19–55, 2019.
- [125] C. Schwab and J. Zech. Deep learning in high dimension: Neural network expression rates for analytic functions in $L^2(\mathbb{R}^d, \gamma_d)$. *SIAM/ASA Journal on Uncertainty Quantification*, 11(1):199–234, 2023.
- [126] J. Seidman, G. Kissas, P. Perdikaris, and G. J. Pappas. NOMAD: Nonlinear manifold decoders for operator learning. In *Thirty-sixth Annual Conference on Neural Information Processing Systems*, 2022.
- [127] N. Sharp, S. Attaiiki, K. Crane, and M. Ovsjanikov. Diffusionnet: Discretization agnostic learning on surfaces. *ACM Transactions on Graphics*, 2022.
- [128] Z. Shen, H. Yang, and S. Zhang. Nonlinear approximation via compositions. *Neural Networks*, 119:74–84, 2019.
- [129] Z. Shen, H. Yang, and S. Zhang. Neural network approximation: Three hidden layers are enough. *Neural Networks*, 141:160–173, 2021.
- [130] J. W. Siegel. Optimal approximation rates for deep ReLU neural networks on Sobolev and Besov spaces. *Journal of Machine Learning Research*, 24(357):1–52, 2023.
- [131] A. Stanzola, S. R. Arridge, B. T. Cox, and B. E. Treeby. A Helmholtz equation solver using unsupervised learning: Application to transcranial ultrasound. *Journal of Computational Physics*, 441:110430, 2021.
- [132] G. Stepaniants. Learning partial differential equations in reproducing kernel hilbert spaces. *Journal of Machine Learning Research*, 24(86):1–72, 2023.
- [133] A. M. Stuart. Inverse problems: A Bayesian perspective. *Acta Numerica*, 19:451–559, 2010.
- [134] R. Swischuk, L. Mainini, B. Peherstorfer, and K. Willcox. Projection-based model reduction: Formulations for physics-based machine learning. *Computers & Fluids*, 179:704–717, 2019.
- [135] T. Tripura and S. Chakraborty. Wavelet neural operator for solving parametric partial differential equations in computational mechanics problems. *Computer Methods in Applied Mechanics and Engineering*, 404:115783, 2023.
- [136] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin. Attention is all you need. In *Thirty-first Annual Conference on Neural Information Processing Systems*, 2017.
- [137] C. Wang and A. Townsend. Operator learning for hyperbolic partial differential equations. *arXiv preprint arXiv:2312.17489*, 2023.

- [138] D. Wang, Z. Zhao, Y. Yu, and R. Willett. Functional linear regression with mixed predictors. *The Journal of Machine Learning Research*, 23(1):12181–12274, 2022.
- [139] J.-X. Wang, J.-L. Wu, and H. Xiao. Physics-informed machine learning approach for reconstructing Reynolds stress modeling discrepancies based on DNS data. *Phys. Rev. Fluids*, 2:034603, 2017.
- [140] J.-L. Wu, H. Xiao, and E. Paterson. Physics-informed machine learning approach for augmenting turbulence models: A comprehensive framework. *Phys. Rev. Fluids*, 3:074602, 2018.
- [141] K. Xu, D. Z. Huang, and E. Darve. Learning constitutive relations using symmetric positive definite neural networks. *Journal of Computational Physics*, 428:110072, 2021.
- [142] D. Yarotsky. Error bounds for approximations with deep ReLU networks. *Neural Networks*, 94:103–114, 2017.
- [143] D. Yarotsky. Elementary superexpressive activations. In *Thirty-eighth International Conference on Machine Learning*, 2021.
- [144] D. Yarotsky and A. Zhevnerchuk. The phase diagram of approximation rates for deep neural networks. In *Thirty-fourth Annual Conference on Neural Information Processing Systems*, 2020.
- [145] E. Yeung, S. Kundu, and N. Hodas. Learning deep neural network representations for Koopman operators of nonlinear dynamical systems. In *2019 American Control Conference (ACC)*, 2019.
- [146] L. Yi, H. Su, X. Guo, and L. J. Guibas. Syncspecnn: Synchronized spectral CNN for 3D shape segmentation. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2017.
- [147] O. Zahm, P. G. Constantine, C. Prieur, and Y. M. Marzouk. Gradient-based dimension reduction of multivariate vector-valued functions. *SIAM Journal on Scientific Computing*, 42(1):A534–A558, 2020.
- [148] S. Zhang, Z. Shen, and H. Yang. Neural network architecture beyond width and depth. In *Thirty-sixth Annual Conference on Neural Information Processing Systems*, 2022.
- [149] Z. Zhang, L. Tat, and H. Schaeffer. BelNet: Basis enhanced learning, a mesh-free neural operator. *Proceedings of the Royal Society A*, 479, 2023.